CS5493 Topics in Autonomous Driving

香港城市大學
City University of Hong Kong
Innovating into the Future

Chapter 5: Planning

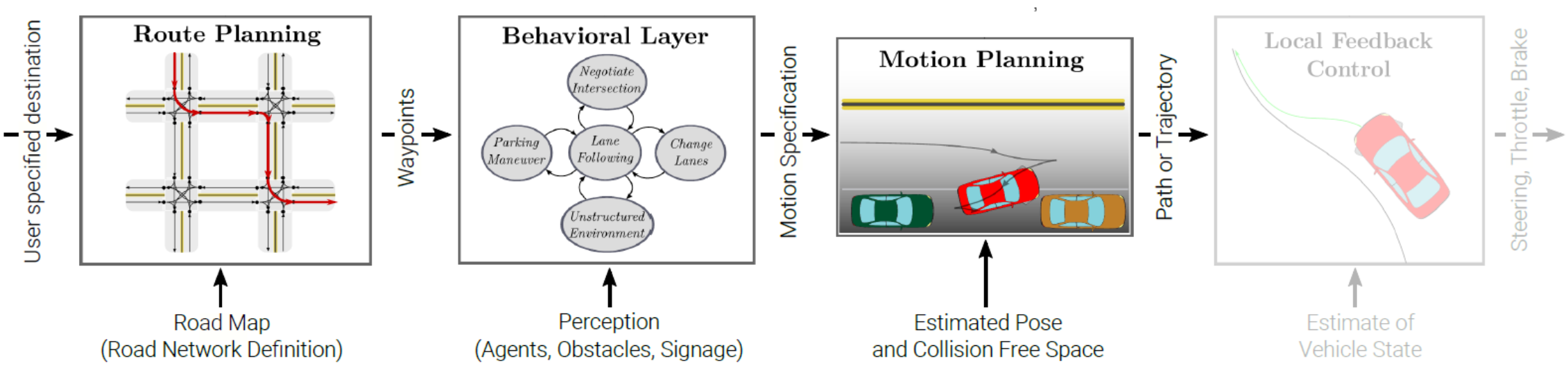
Planning

- Goal: find and follow path from current location to destination
- Take static infrastructure and dynamic objects into account
- Input: vehicle and environment states and prediction
- Output: path or trajectory as input to vehicle controller

- Challenges:
  - Driving situations and behaviors are very complex
  - Difficult to model as a single optimization problem

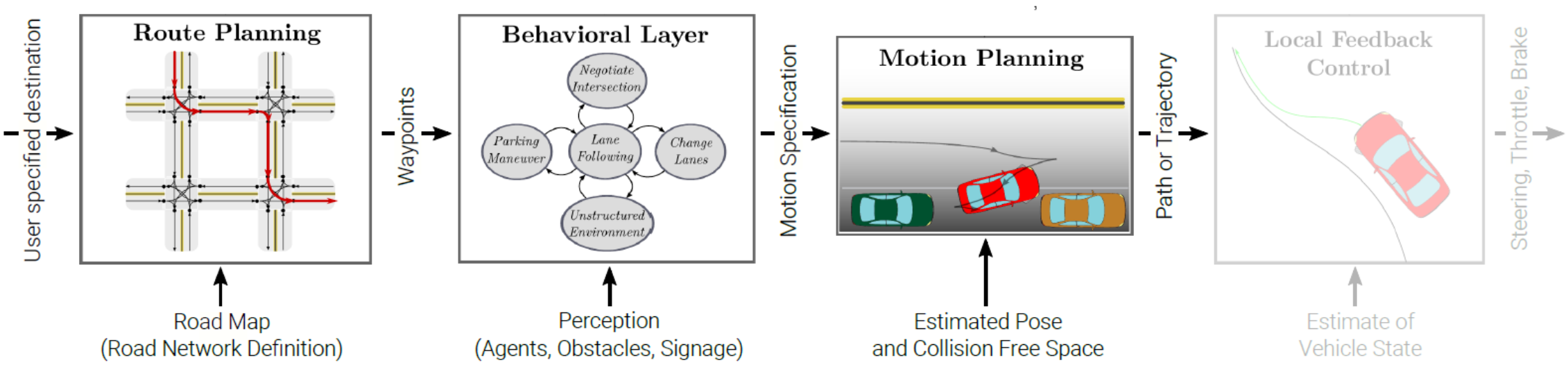
Planning

- Break planning problem into a **hierarchy of simpler problems**
  - Each problem tailored to its scope and level of abstraction
  - Earlier in this hierarchy means higher level of abstraction
  - Each optimization problem will have constraints and objective functions



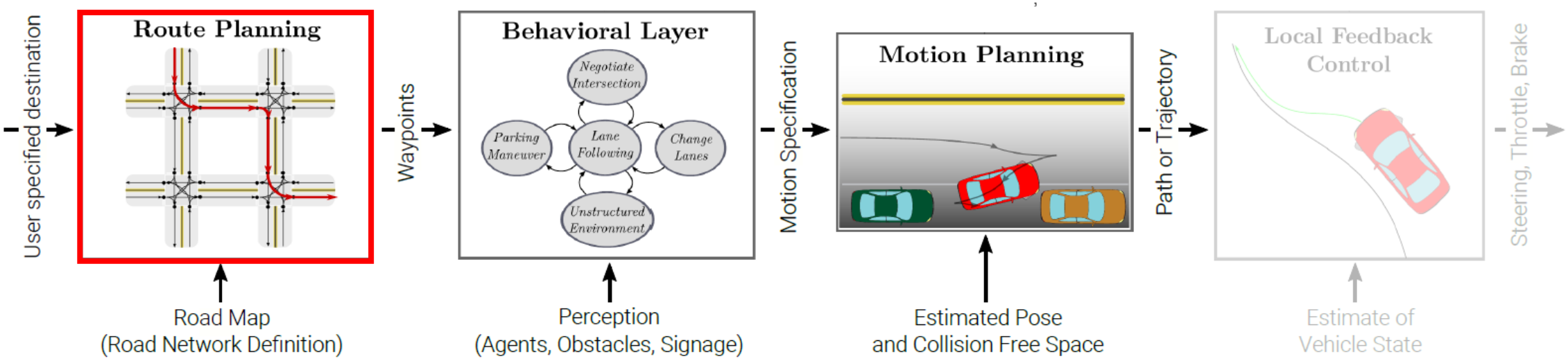
Planning

- Break planning problem into a **hierarchy of simpler problems**
  - **Route planning**: generates a route through the road network.
  - **Behavioral planning:** reasons about the environment and generates a motion specification to progress along the selected route.
  - **Motion planning:** solves for a feasible motion accomplishing the specification.



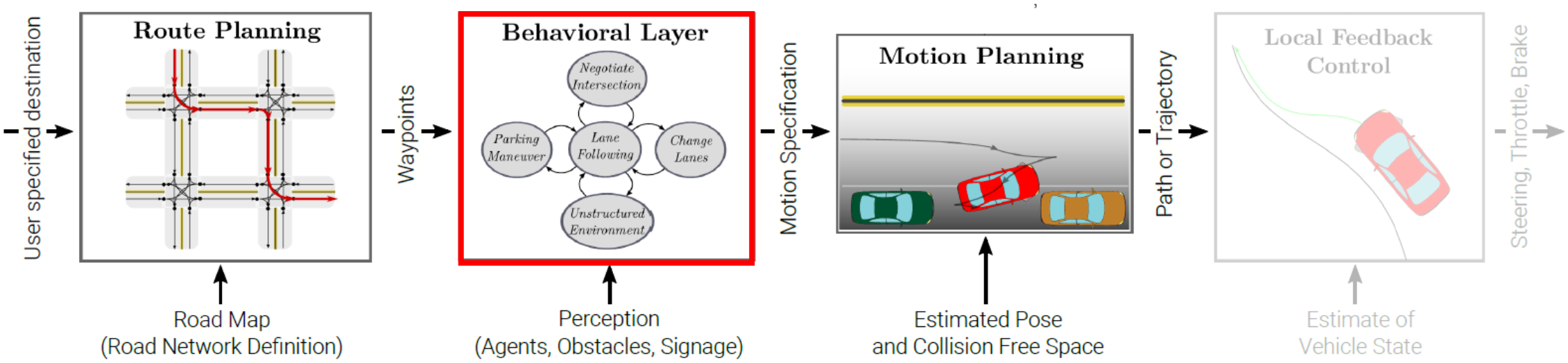
Route Planning

- Represent **road network** as **directed graph**
- Edge weights correspond to road segment length or travel time
- Problem translates into a minimum-cost graph network problem
- Inference algorithms: Dijkstra, A*, . . .



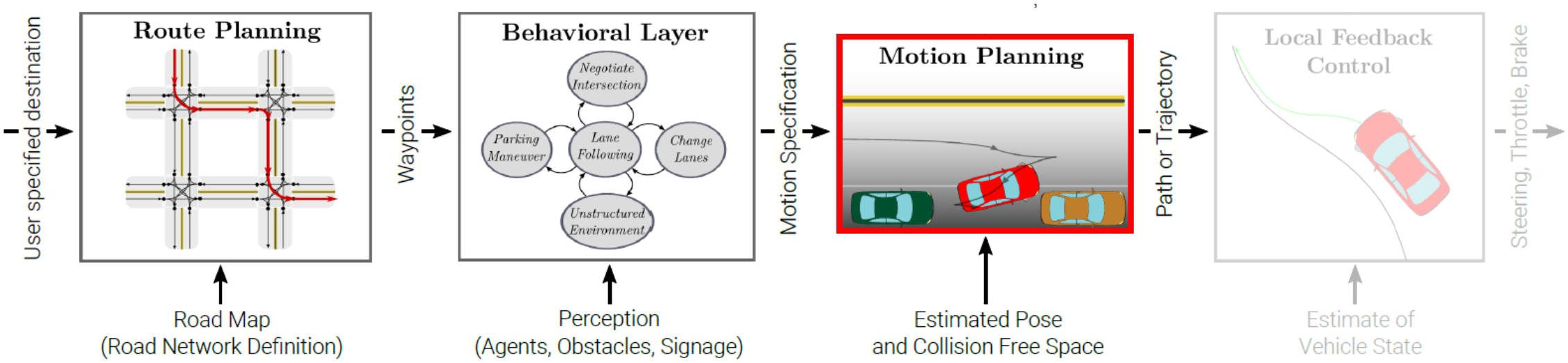
Behavior Planning

- Select **driving behavior** based on current vehicle/environment state
- E.g. at stop line: stop, observe other traffic participants, traverse
- Often modeled via **finite state machines** (transitions governed by perception)
- Can be modeled probabilistically, e.g., using Markov Decision Processes (MDPs)



Motion Planning

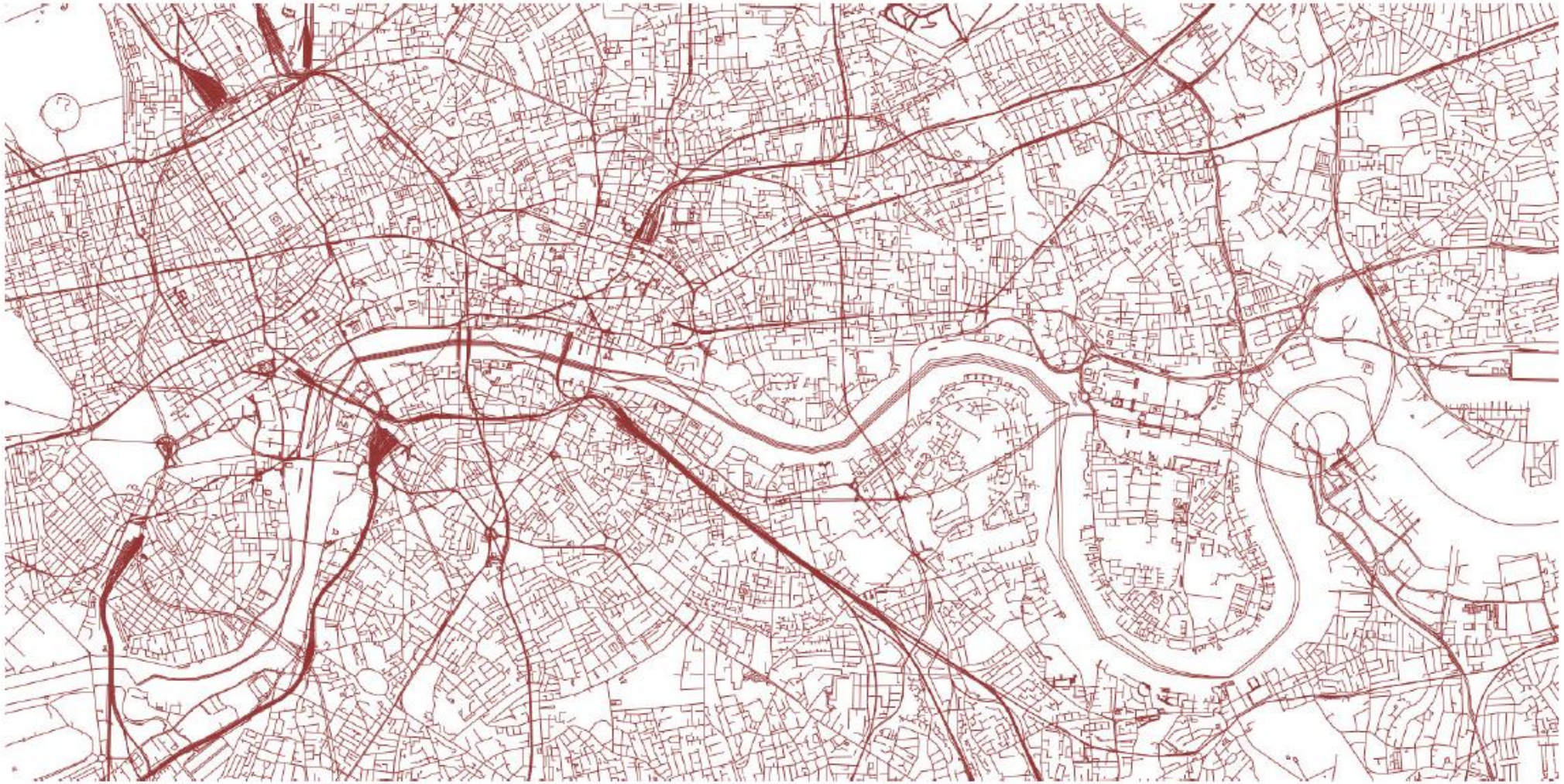
- Find feasible, comfortable, safe and fast **vehicle path/trajectory**
- Exact solutions in most cases computationally intractable
- Thus often **numerical approximations** are used
- Approaches: variational methods, graph search, incremental tree-based



Route Planning

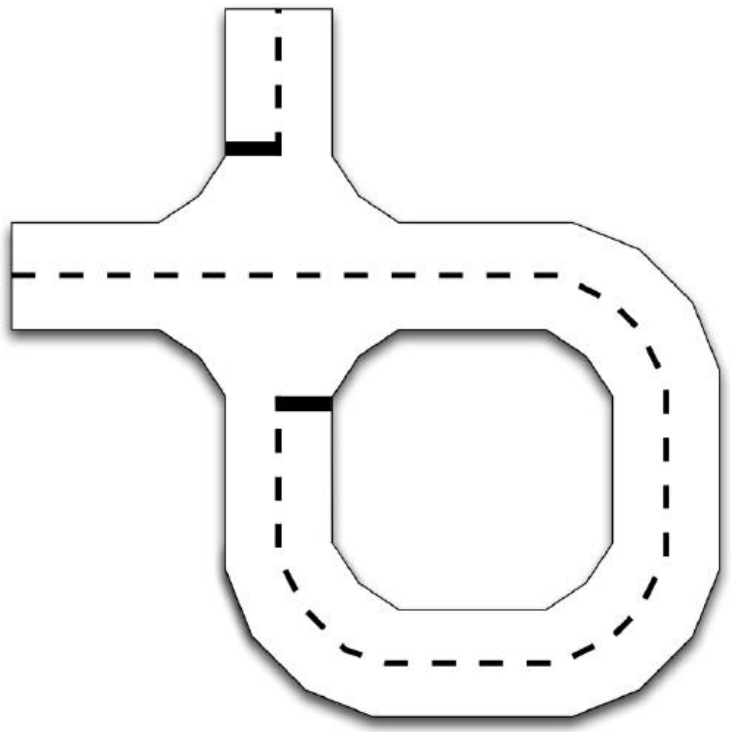
- Navigate from point A to point B on a road map using the **most efficient path** (in terms of time or distance)
- Route planning is **high-level planning**
- Low-level details are abstracted away
- Route planning is sometimes also referred to as “mission planning”

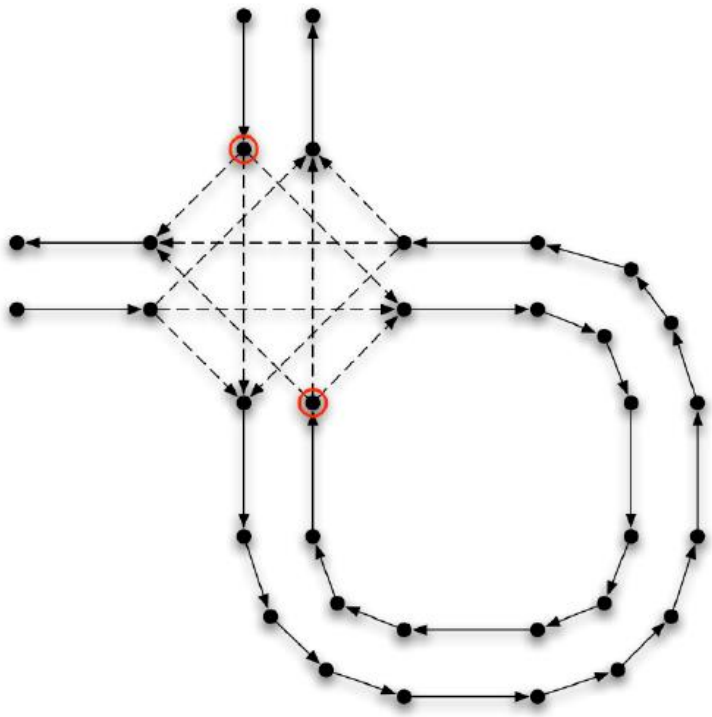
Road Networks as Graphs



Road Networks as Graphs

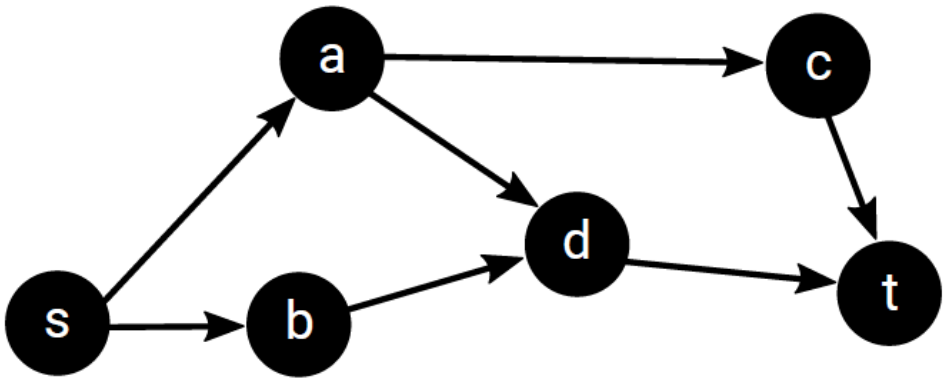
- Road networks can be represented as **directed graphs**





Road Networks as Graphs

- Road networks can be represented as **directed graphs**
- Edges correspond to road or lane segments
- We are interested in finding the **shortest path** from one node to another



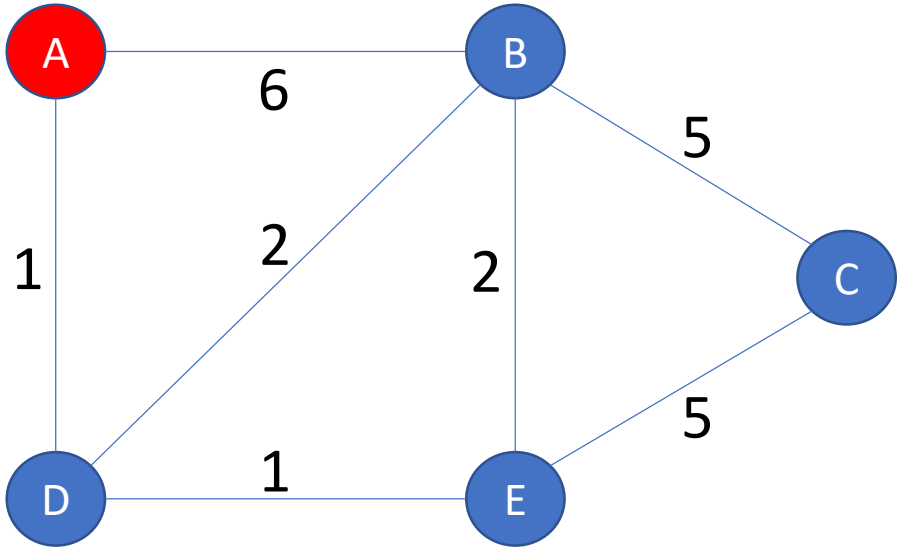
Dijkstra's Shortest Path Algorithm

- A greedy algorithm to find the shortest path from a single source node to all other nodes in a weighted graph (non-negative edge weights only)

- Time Complexity
  - Basic (Array): $O(V^2)$ (V = number of nodes)
  - Optimized (Min-Heap): $O((V + E) \log V)$ (E = number of edges)

Dijkstra's Shortest Path Algorithm

- Consider the start vertex, A
- Distance to A from A = 0
- Distances to all other vertices from A are unknown, therefore infinity



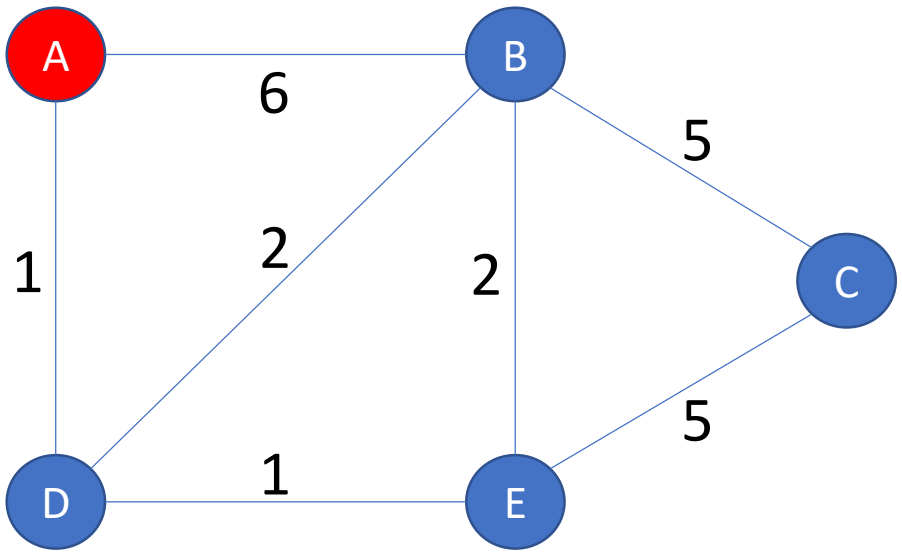
Vertex	Shortest distance from A	Previous vertex
A		
B		
C		
D		
E		

Visited = []

Unvisited = [A, B, C, D, E]

Dijkstra's Shortest Path Algorithm

- Visit the unvisited vertex with the smallest known distance from the start vertex
- At this step, it is the start vertex itself, A



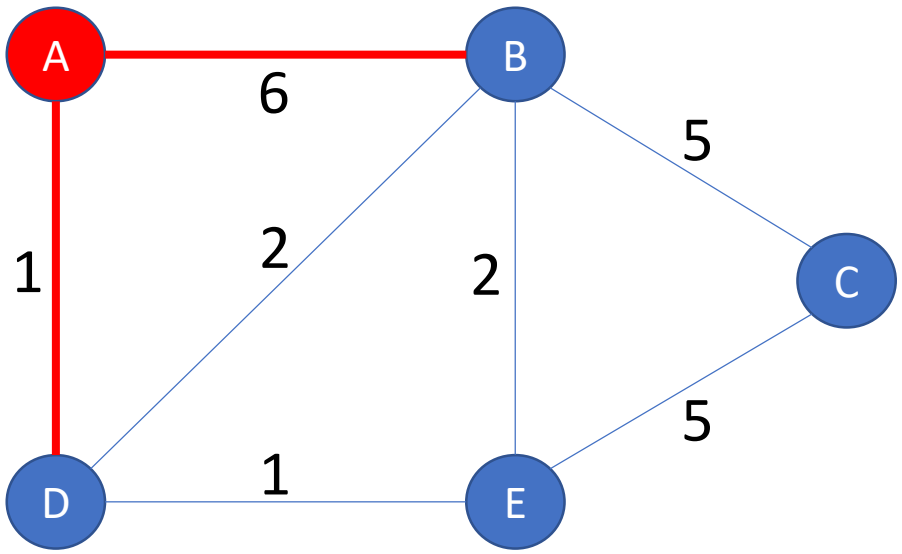
Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

Visited = []

Unvisited = [A, B, C, D, E]

Dijkstra's Shortest Path Algorithm

- For the current vertex, examine its unvisited neighbors
- We are currently visiting A and its unvisited neighbors are B and D



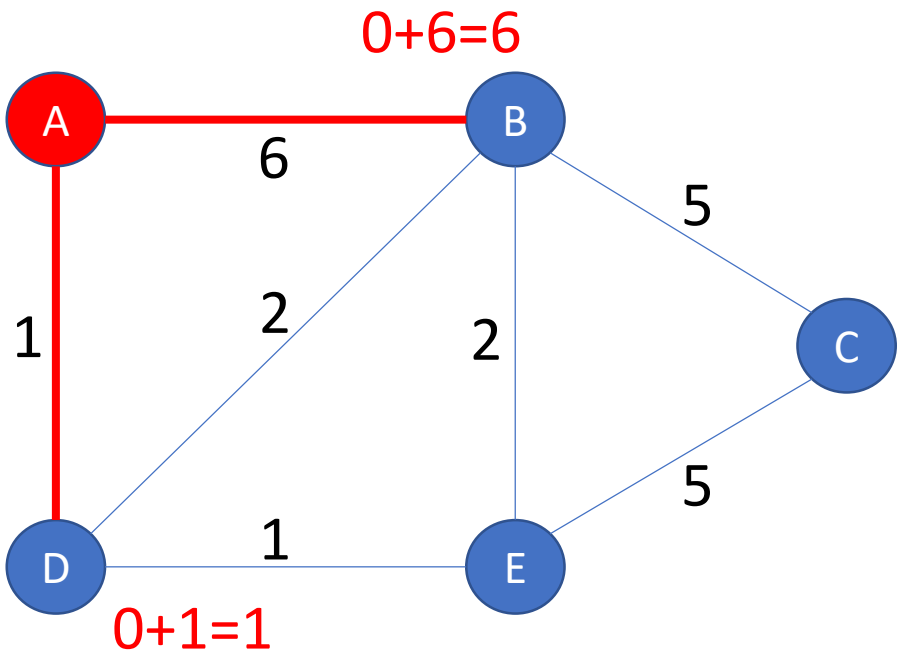
Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

Visited = []

Unvisited = [A, B, C, D, E]

Dijkstra's Shortest Path Algorithm

- For the current vertex, calculate the distance of each neighbor from the start vertex



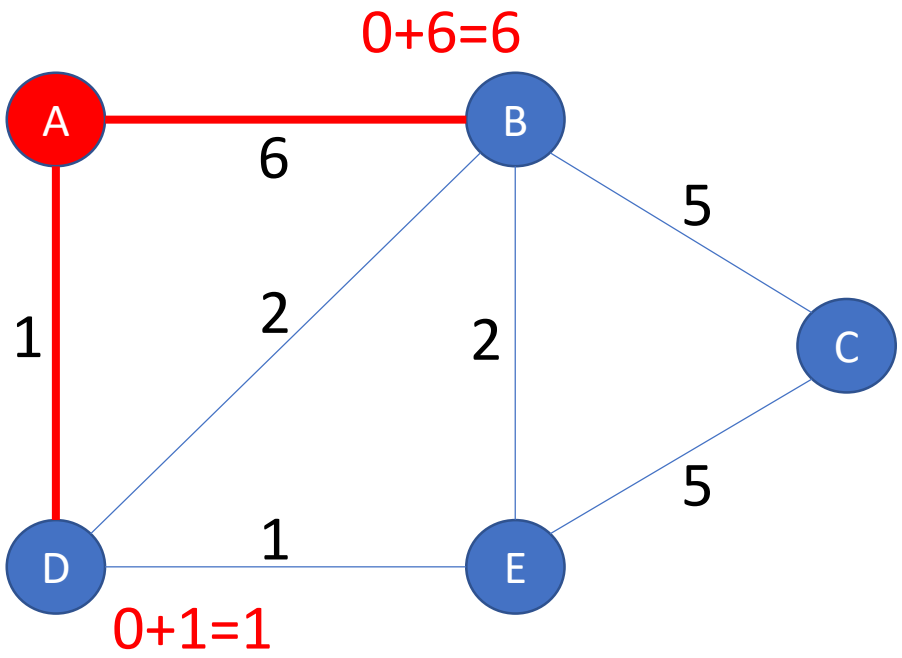
Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

Visited = []

Unvisited = [A, B, C, D, E]

Dijkstra's Shortest Path Algorithm

- If the calculated distance of a vertex is less than the known distance, update the shortest distance



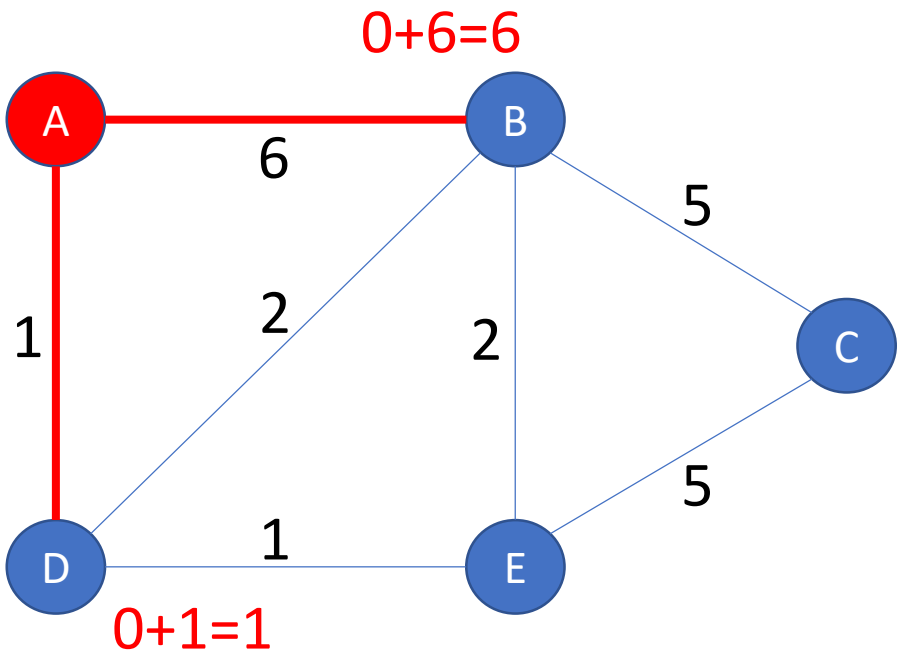
Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

Visited = []

Unvisited = [A, B, C, D, E]

Dijkstra's Shortest Path Algorithm

- Update the previous vertex for each of the updated distances
- In this case, we visited B and D via A



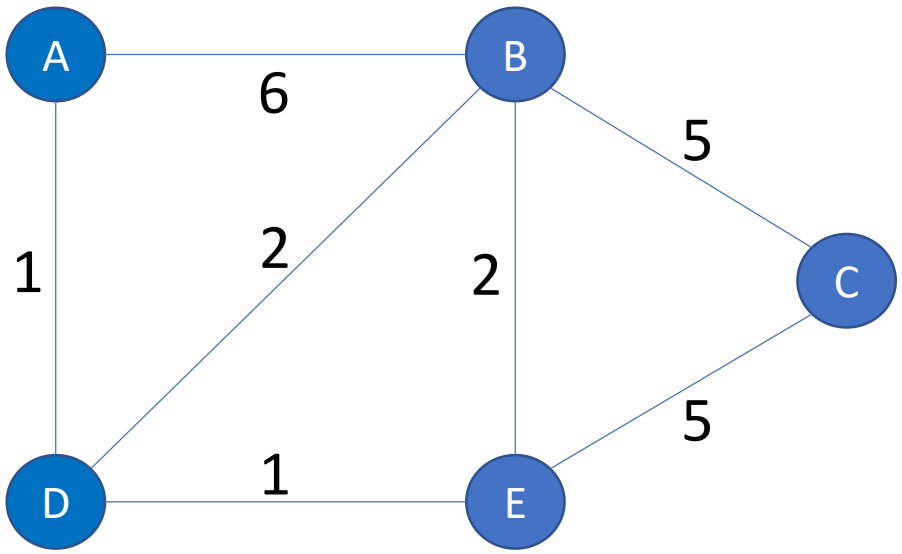
Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**6**	
C	∞	
D	**1**	
E	∞	

Visited = []

Unvisited = [A, B, C, D, E]

Dijkstra's Shortest Path Algorithm

- Update the previous vertex for each of the updated distances
- In this case, we visited B and D via A



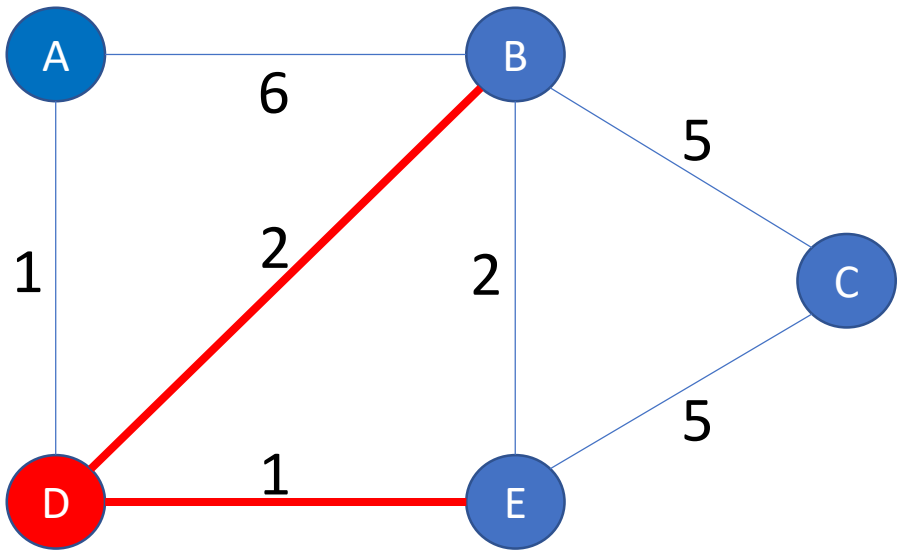
Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**6**	A
C	∞	
D	**1**	A
E	∞	

Visited = [A]

Unvisited = [B, C, D, E]

Dijkstra's Shortest Path Algorithm

- Visit the unvisited vertex with the smallest known distance from the start vertex
- At this step, it is vertex D



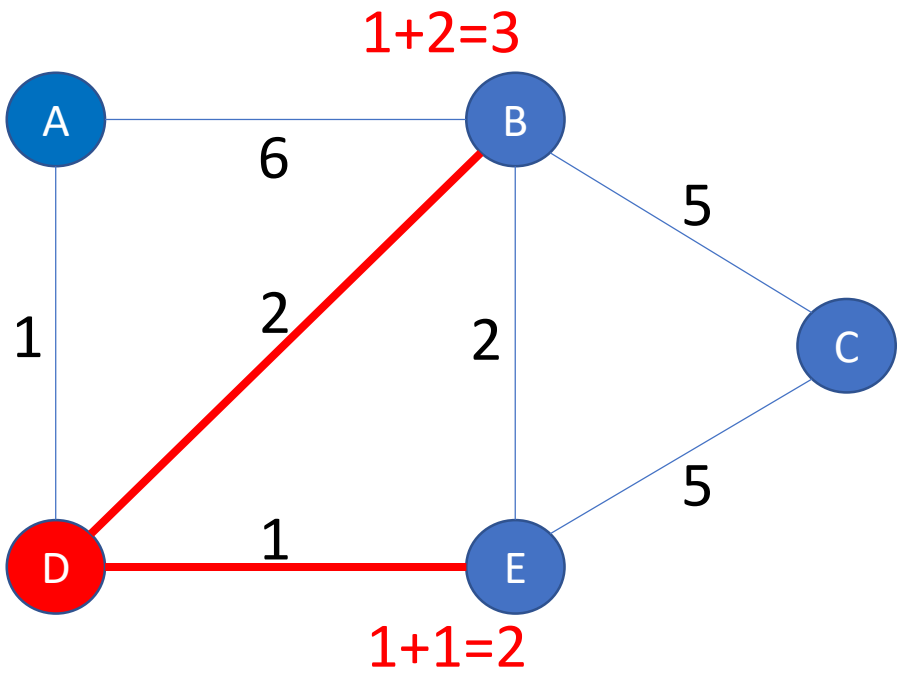
Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**6**	A
C	∞	
D	**1**	A
E	∞	

Visited = [A]

Unvisited = [B, C, D, E]

Dijkstra's Shortest Path Algorithm

- For the current vertex, calculate the distance of each neighbor from the start vertex



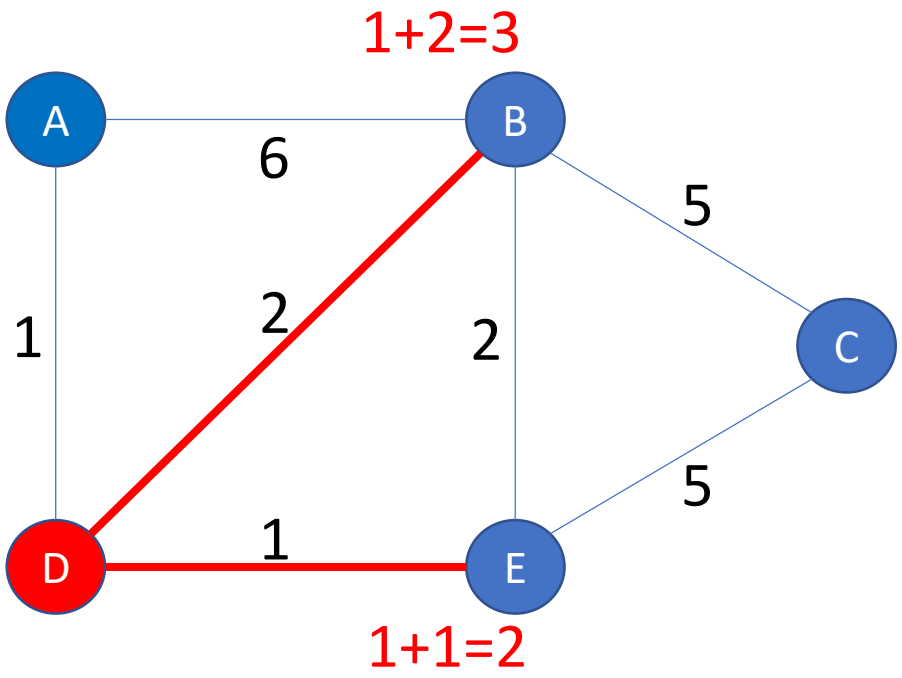
Visited = [A]

Unvisited = [B, C, D, E]

Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**6**	A
C	∞	
D	**1**	A
E	∞	

Dijkstra's Shortest Path Algorithm

- If the calculated distance of a vertex is less than the known distance, update the shortest distance



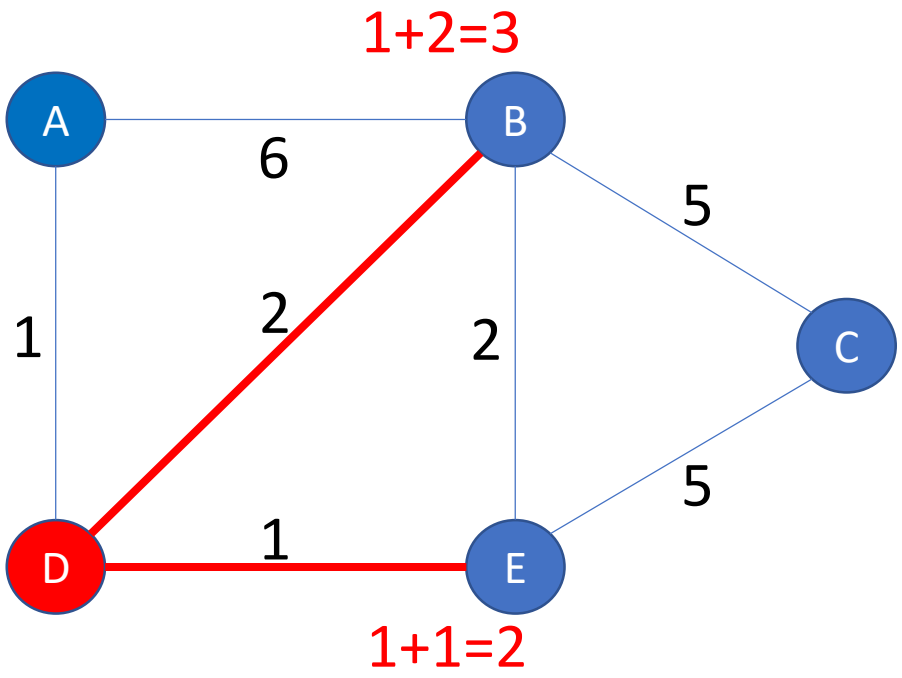
Visited = [A]

Unvisited = [B, C, D, E]

Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	A
C	∞	
D	**1**	A
E	**2**	

Dijkstra's Shortest Path Algorithm

- Update the previous vertex for each of the updated distances
- In this case, we visited B and E via D



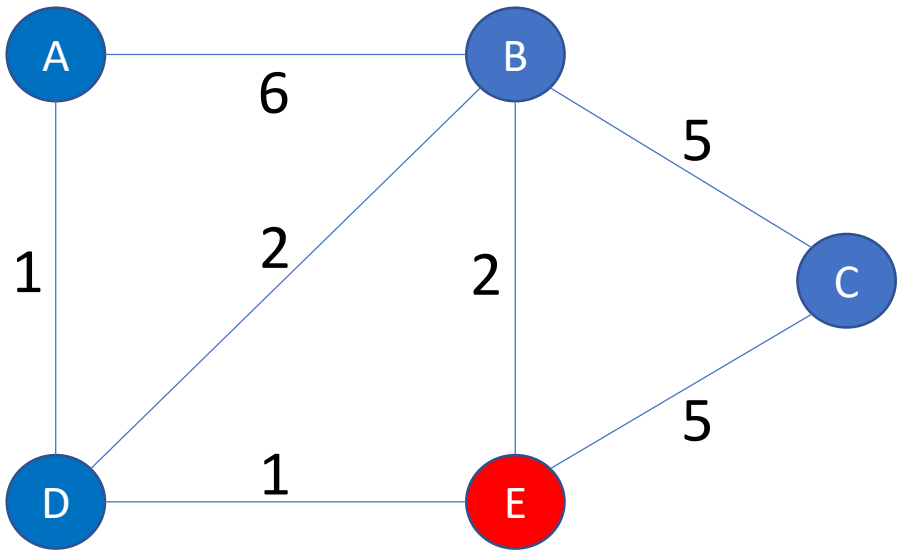
Visited = [A, D]

Unvisited = [B, C, D, E]

Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	∞	
D	**1**	A
E	**2**	D

Dijkstra's Shortest Path Algorithm

- Visit the unvisited vertex with the smallest known distance from the start vertex
- At this step, it is vertex E



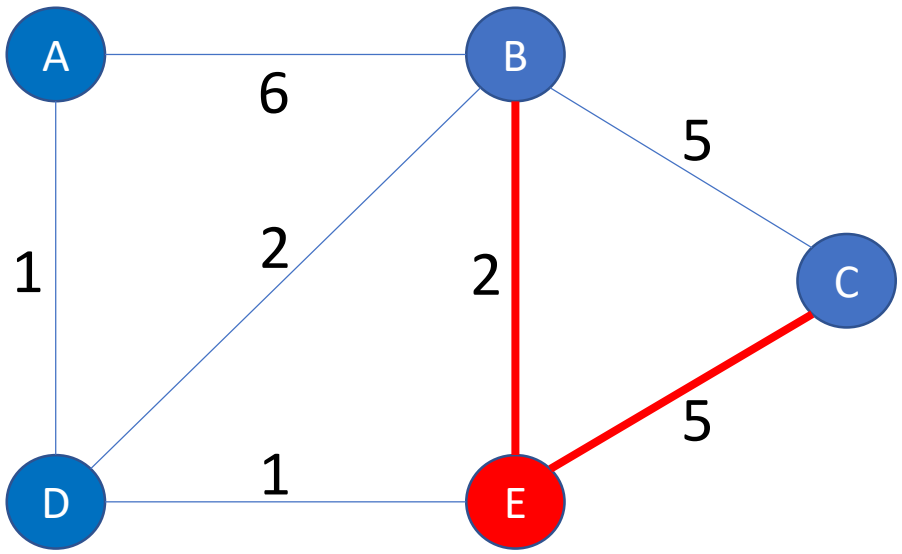
Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	∞	
D	**1**	A
E	2	D

Visited = [A, D]

Unvisited = [B, C, E]

Dijkstra's Shortest Path Algorithm

- For the current vertex, examine its unvisited neighbors
- We are currently visiting E and its unvisited neighbors are B and C



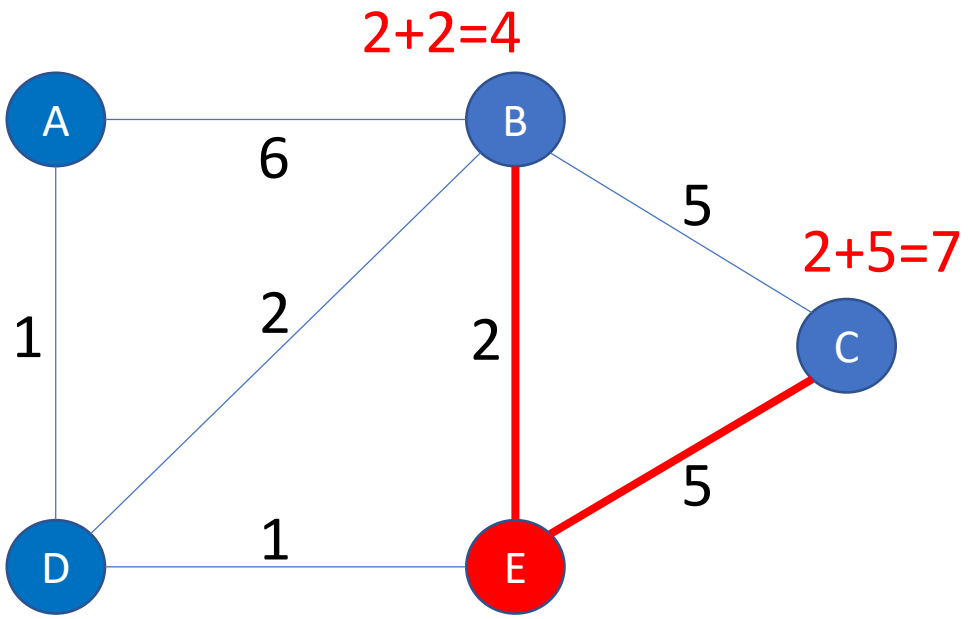
Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	∞	
D	**1**	A
E	2	D

Visited = [A, D]

Unvisited = [B, C, E]

Dijkstra's Shortest Path Algorithm

- For the current vertex, calculate the distance of each neighbor from the start vertex



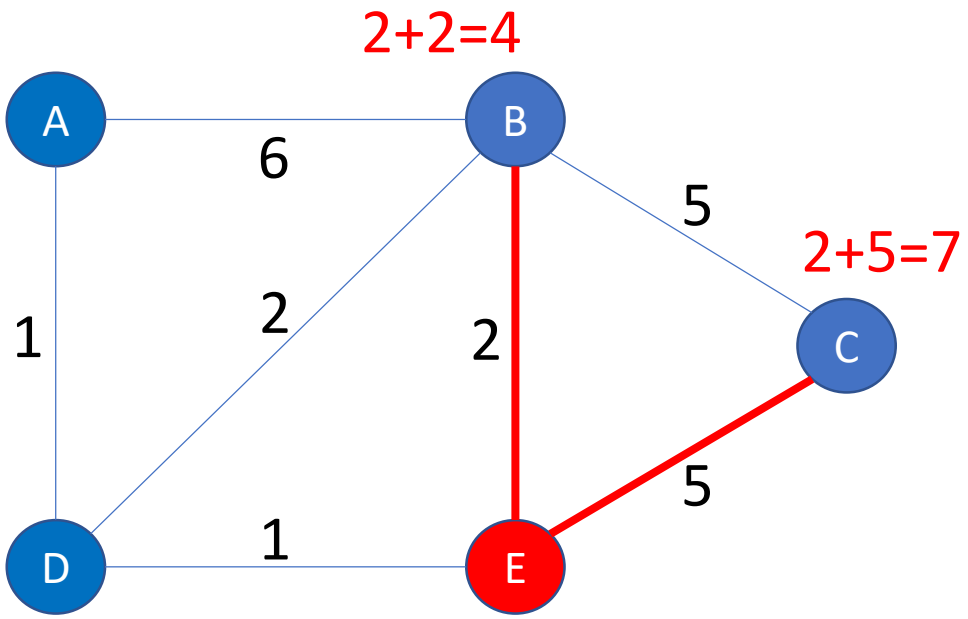
Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	∞	
D	**1**	A
E	2	D

Visited = [A, D]

Unvisited = [B, C, E]

Dijkstra's Shortest Path Algorithm

- If the calculated distance of a vertex is less than the known distance, update the shortest distance



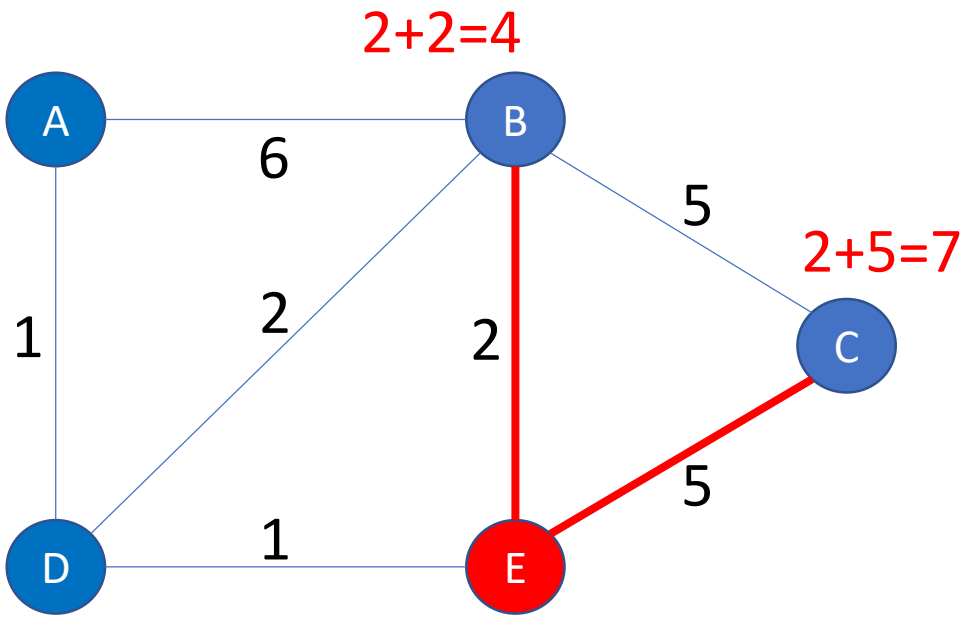
Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	
D	**1**	A
E	2	D

Visited = [A, D]

Unvisited = [B, C, E]

Dijkstra's Shortest Path Algorithm

- Update the previous vertex for each of the updated distances
- In this case, we visited C via E

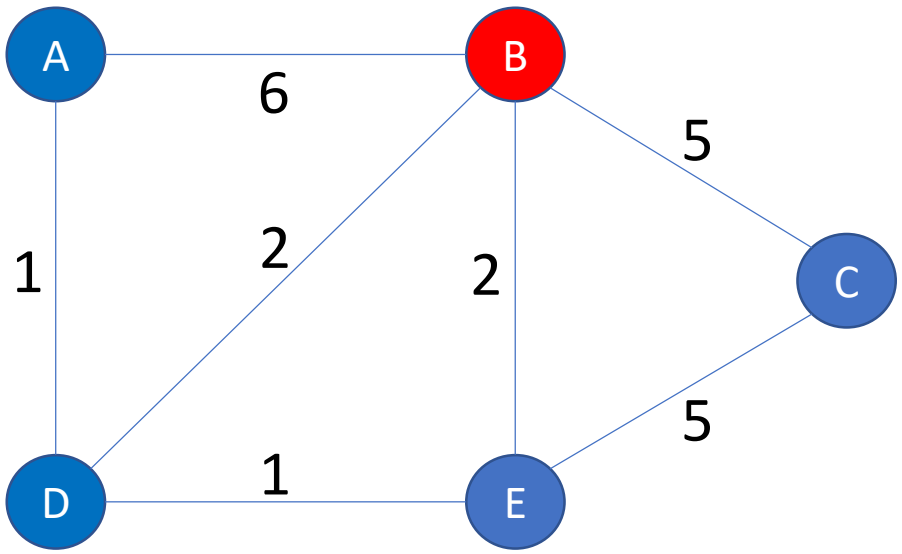


Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	E
D	**1**	A
E	2	D

Visited = [A, D, E] Unvisited = [B, C]

Dijkstra's Shortest Path Algorithm

- Visit the unvisited vertex with the smallest known distance from the start vertex
- At this step, it is vertex B

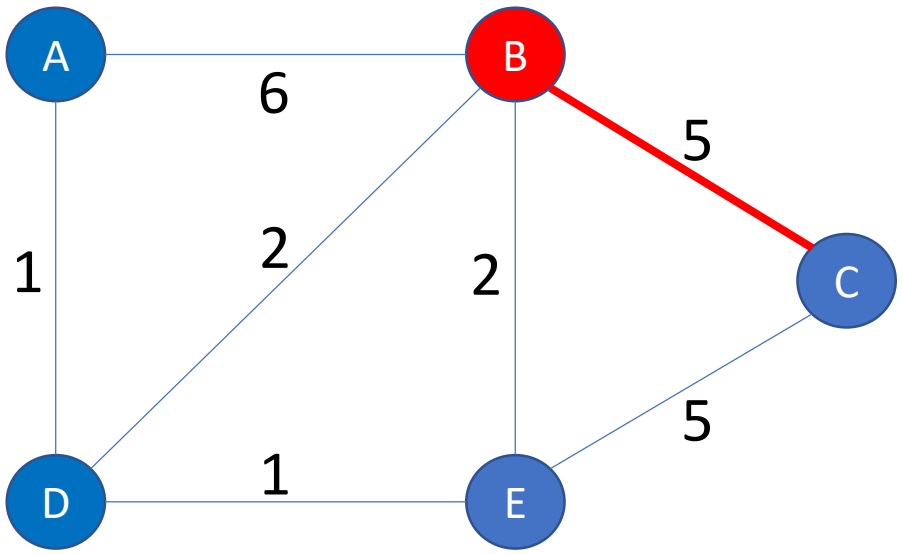


Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	E
D	**1**	A
E	2	D

Visited = [A, D, E] Unvisited = [B, C]

Dijkstra's Shortest Path Algorithm

- For the current vertex, examine its unvisited neighbors
- We are currently visiting B and its only unvisited neighbors is C

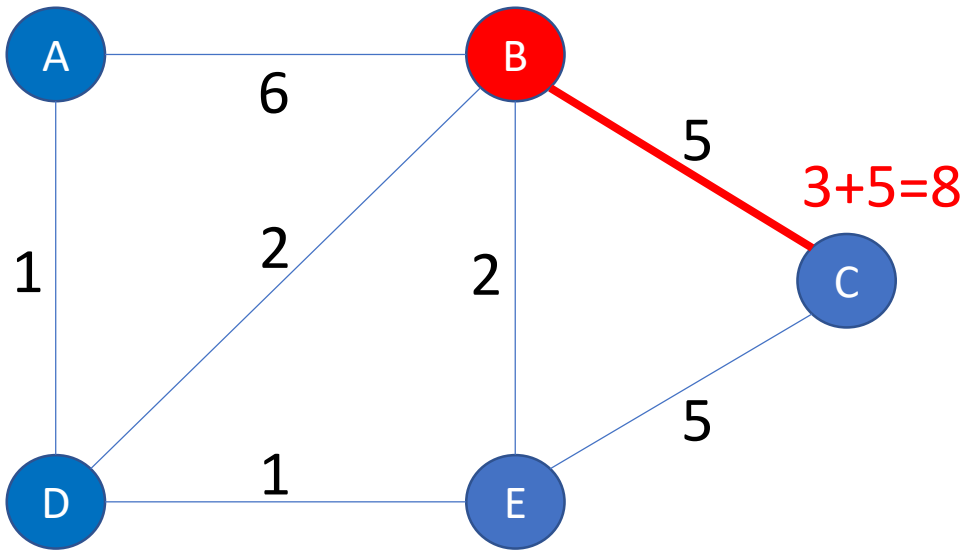


Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	E
D	**1**	A
E	2	D

Visited = [A, D, E] Unvisited = [B, C]

Dijkstra's Shortest Path Algorithm

- For the current vertex, calculate the distance of each neighbor from the start vertex

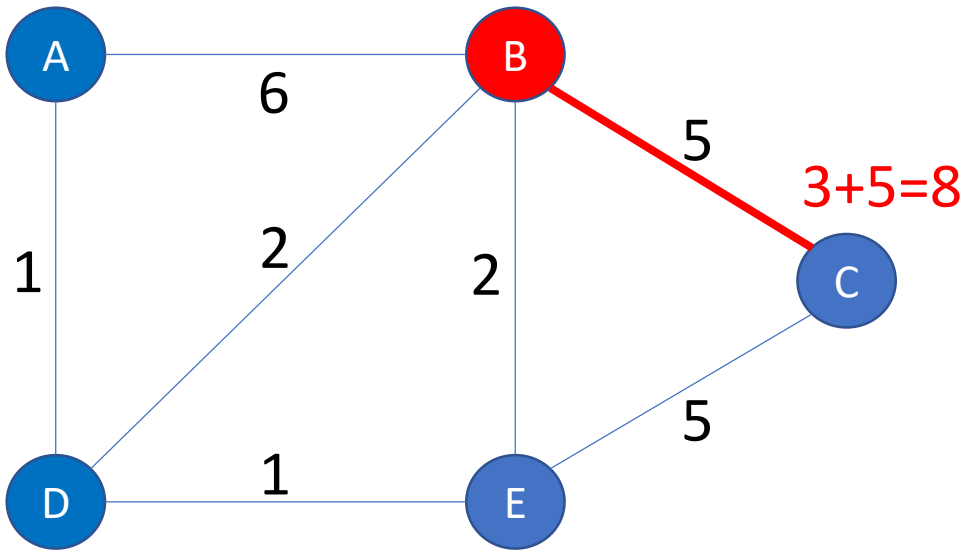


Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	E
D	**1**	A
E	2	D

Visited = [A, D, E] Unvisited = [B, C]

Dijkstra's Shortest Path Algorithm

- If the calculated distance of a vertex is less than the known distance, update the shortest distance

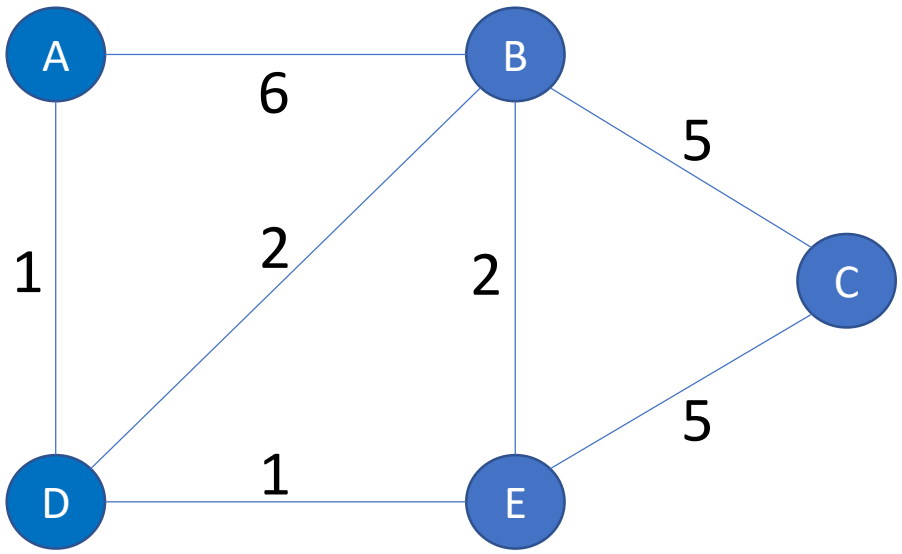


Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	E
D	**1**	A
E	2	D

Visited = [A, D, E] Unvisited = [B, C]

Dijkstra's Shortest Path Algorithm

- If the calculated distance of a vertex is less than the known distance, update the shortest distance

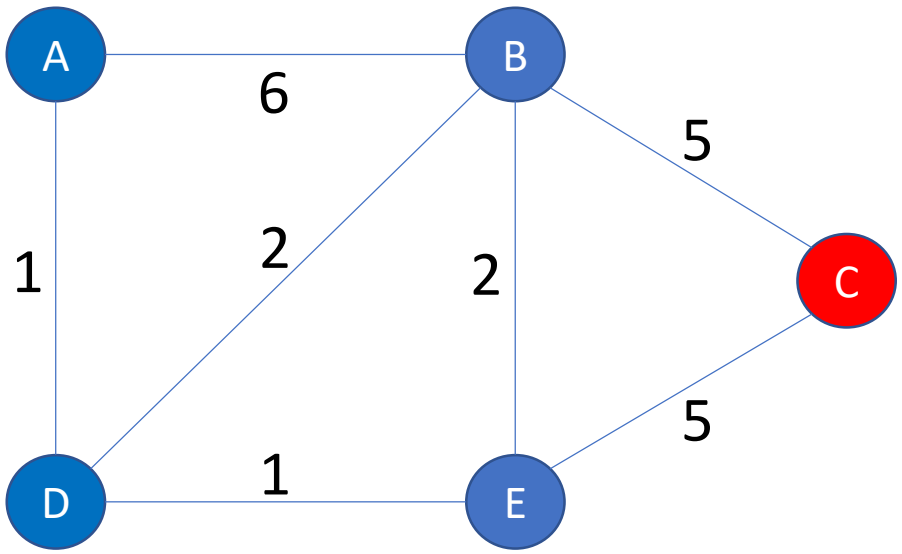


Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	E
D	**1**	A
E	2	D

Visited = [A, D, E, B] Unvisited = [C]

Dijkstra's Shortest Path Algorithm

- Visit the unvisited vertex with the smallest known distance from the start vertex
- At this step, it is vertex C

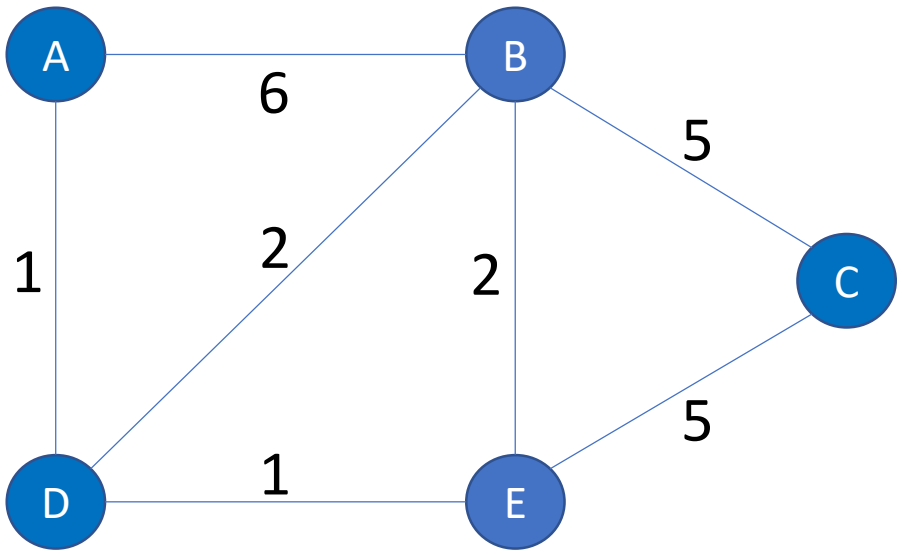


Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	E
D	**1**	A
E	2	D

Visited = [A, D, E, B] Unvisited = [C]

Dijkstra's Shortest Path Algorithm

- For the current vertex, examine its unvisited neighbors
- We are currently visiting C and it has no unvisited neighbors



Vertex	Shortest distance from A	Previous vertex
A	**0**	
B	**3**	D
C	**7**	E
D	**1**	A
E	2	D

Visited = [A, D, E, B, C] Unvisited = []

Dijkstra’s Shortest Path Algorithm

Let distance of start vertex from start vertex=0

Let distance of all other vertices from start=∞(infinity)

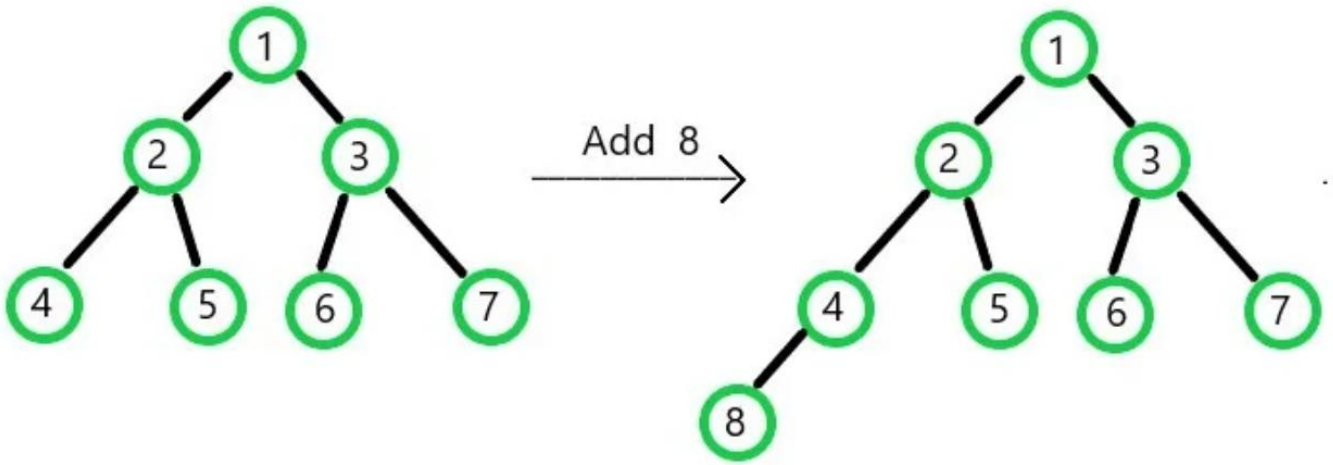
Repeat

- Visit the unvisited vertex with the smallest known distance from the start vertex
- For the current vertex, examine its unvisited neighbors
- For the current vertex, calculate distance of each neighbor from start vertex
- If the calculated distance of a vertex is less than the known distance, update the shortest distance
- Update the previous vertex for each of the updated distances
- Add the current vertex to the list of visited vertices

Until all vertices visited

Dijkstra's Shortest Path Algorithm

- MinHeap: a tree-based data structure with one strict rule: The **parent node** always has a smaller value than its **child nodes**.
  - The smallest value in the entire heap is always at the **top** (root) of the tree.
- Both the **push (insert)** and **pop (extract minimum)** operations in a min-heap have a time complexity of **O(log *n*)**.



Dijkstra's Shortest Path Algorithm

- Pseudo-code

Algorithm 1 DIJKSTRA(G, s, t)

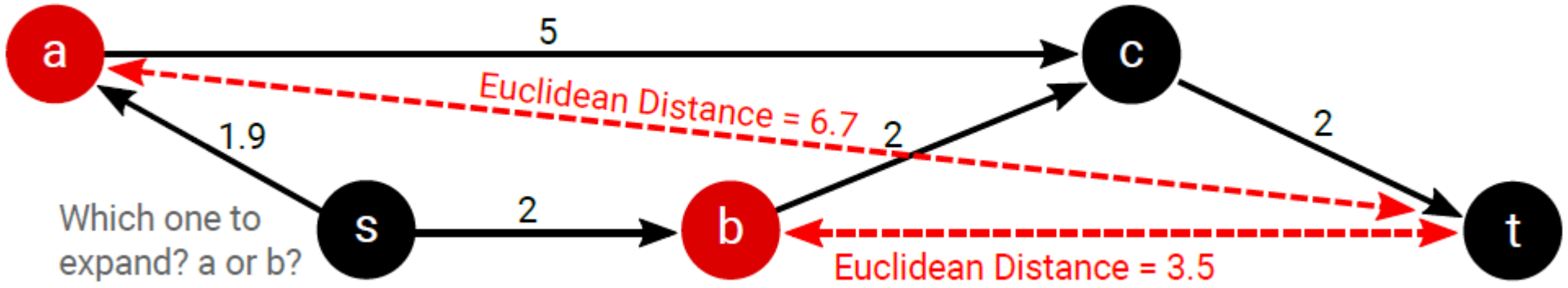
```
open ← MinHeap()
closed ← Set()
predecessors ← Dict()
open.push(s, 0)
while !open.isEmpty() do
    u, uCost ← open.pop()
    if isGoal(u) then
        return extractPath(u, predecessors)
    for all v ∈ u.successors do
        if v ∈ closed then
            continue
        uvCost ← edgeCost(G, u, v)
        if v ∈ open then
            if uCost + uvCost < open[v] then
                open[v] ← uCost + uvCost
                predecessors[v] ← u
        else
            open.push(v, uCost + uvCost)
            predecessors[v] ← u
    closed.add(u)
```

Dijkstra's Shortest Path Algorithm

- The scalability problem
  - Method does not scale to large graphs
  - Computationally expensive/slow
- Example: Map of New York City
  - 54,837 vertices
  - 140,497 edges
- Solution? Use **planning heuristics**
- Example: Use **distance to target** as heuristic to select "promising" vertices to expand

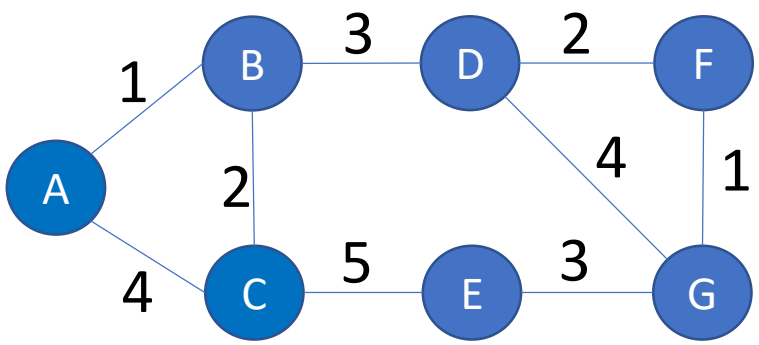
Euclidean Planning Heuristics

- Euclidean planning heuristics **exploits structure of planar graphs**
- Straight line between vertices is a useful estimate of the distance along the path
- The A* algorithm uses this heuristics to plan **more efficiently**
- It always returns the **optimal solution**, independent of the chosen heuristic
  - A better heuristic will make it faster



A* Algorithm

$$f(n) = g(n) + h(n)$$

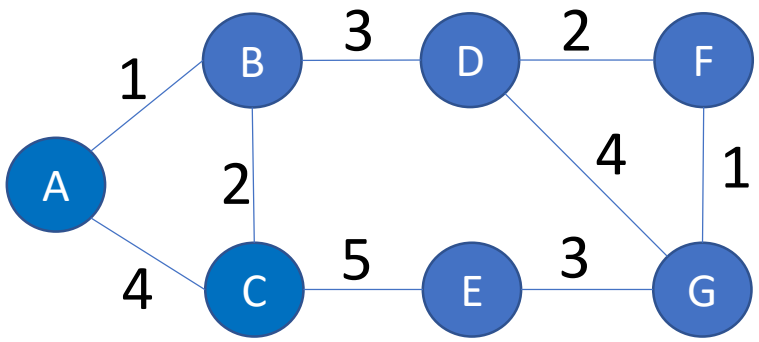


A	B	C	D	E	F	G
5	6	4	3	3	1	0

h(n)

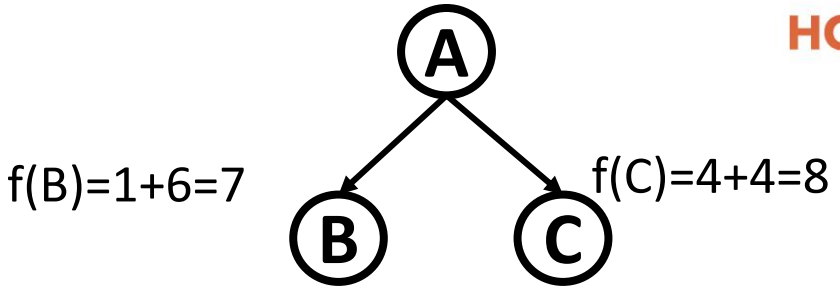
A* Algorithm

$$f(n) = g(n) + h(n)$$



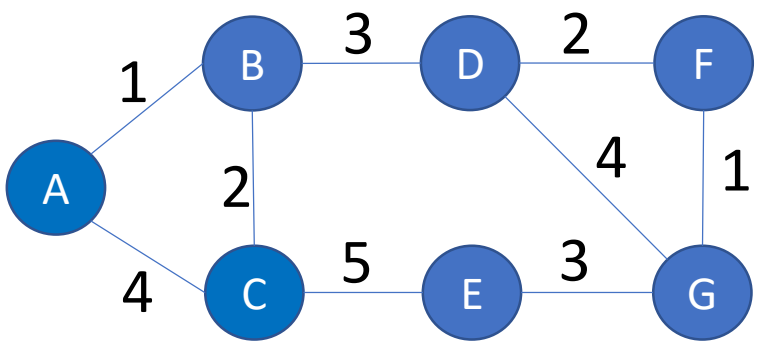
A	B	C	D	E	F	G
5	6	4	3	3	1	0

h(n)



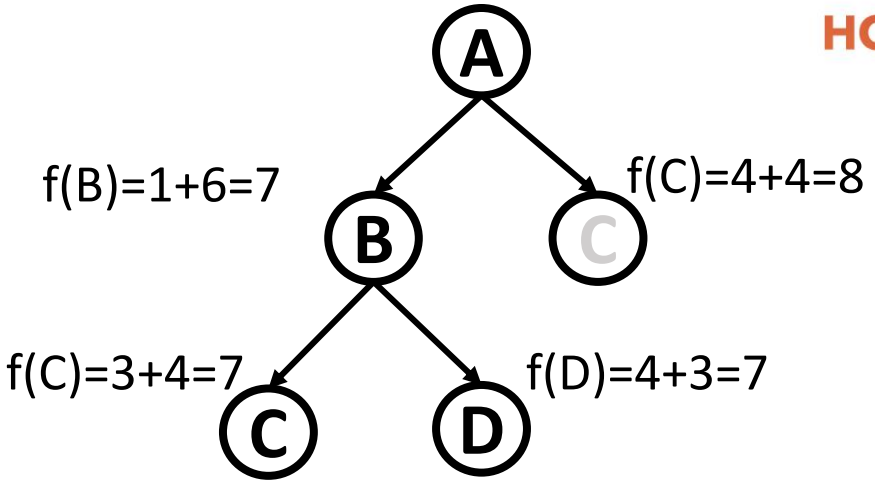
A* Algorithm

$$f(n) = g(n) + h(n)$$



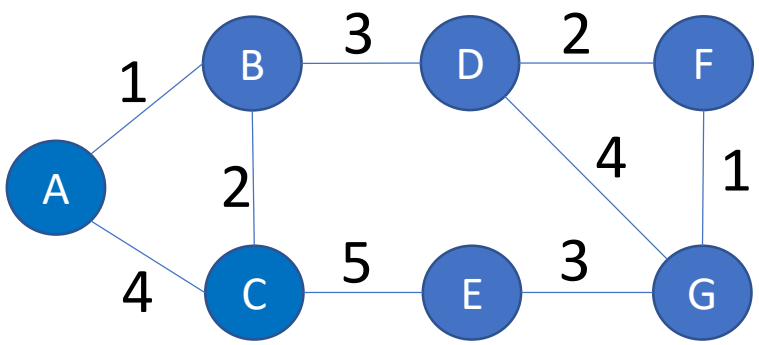
A	B	C	D	E	F	G
5	6	4	3	3	1	0

h(n)



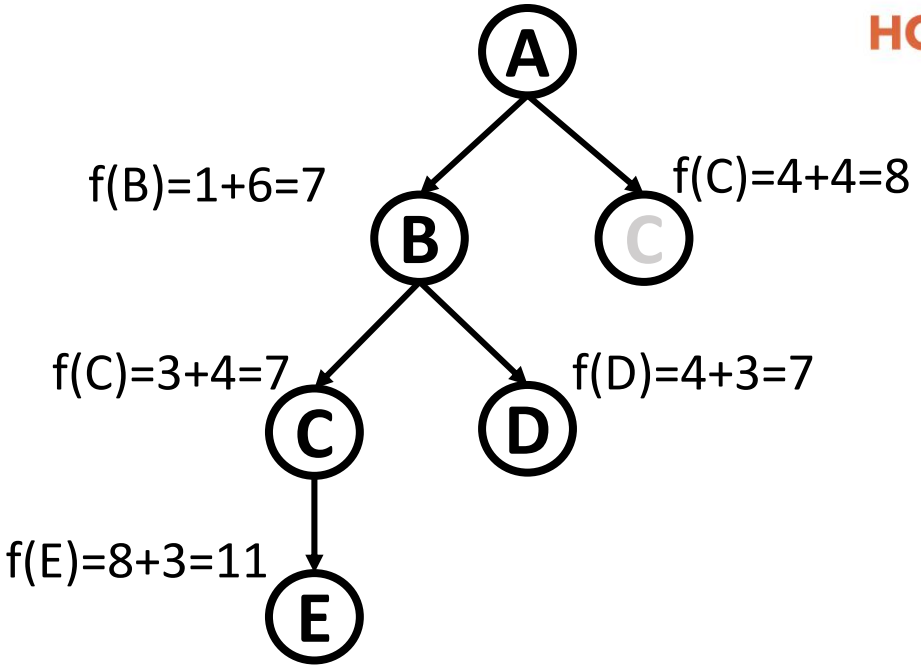
A* Algorithm

$$f(n) = g(n) + h(n)$$



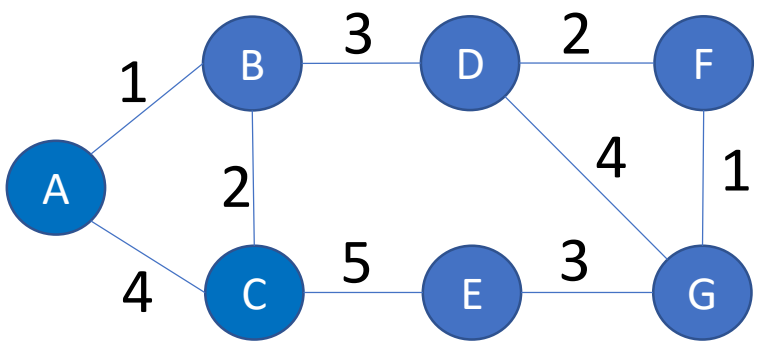
A	B	C	D	E	F	G
5	6	4	3	3	1	0

h(n)



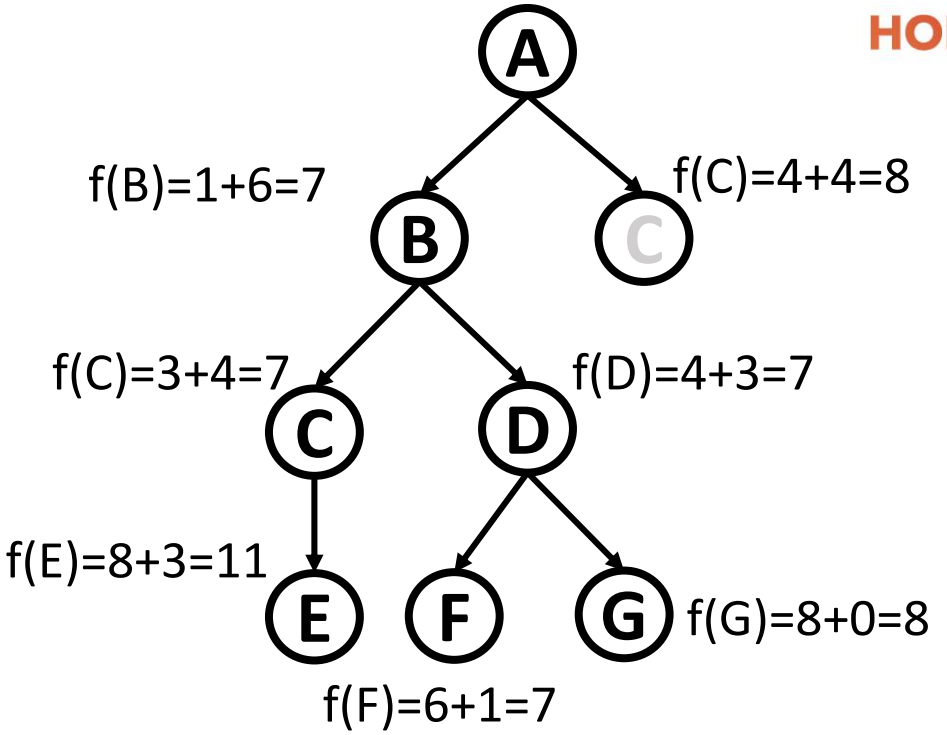
A* Algorithm

$$f(n) = g(n) + h(n)$$



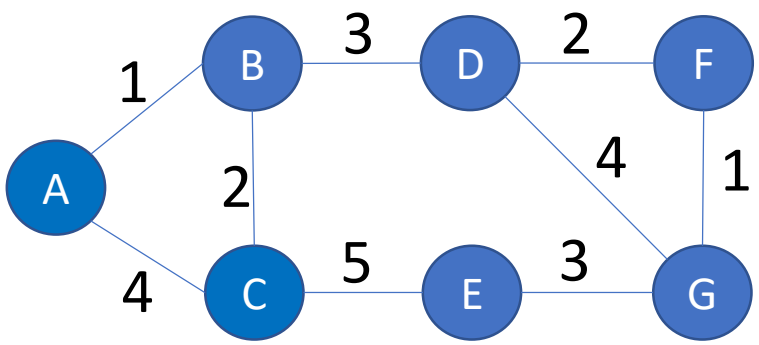
A	B	C	D	E	F	G
5	6	4	3	3	1	0

h(n)



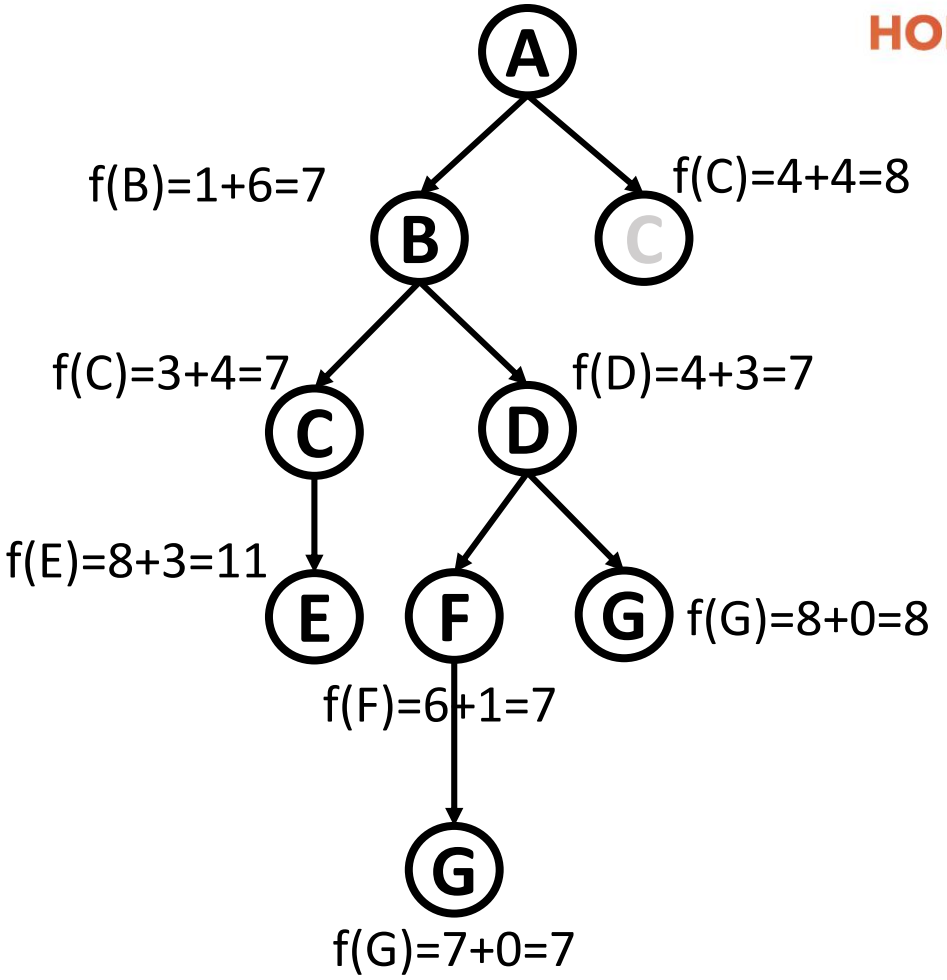
A* Algorithm

$$f(n) = g(n) + h(n)$$



A	B	C	D	E	F	G
5	6	4	3	3	1	0

h(n)



A* Algorithm

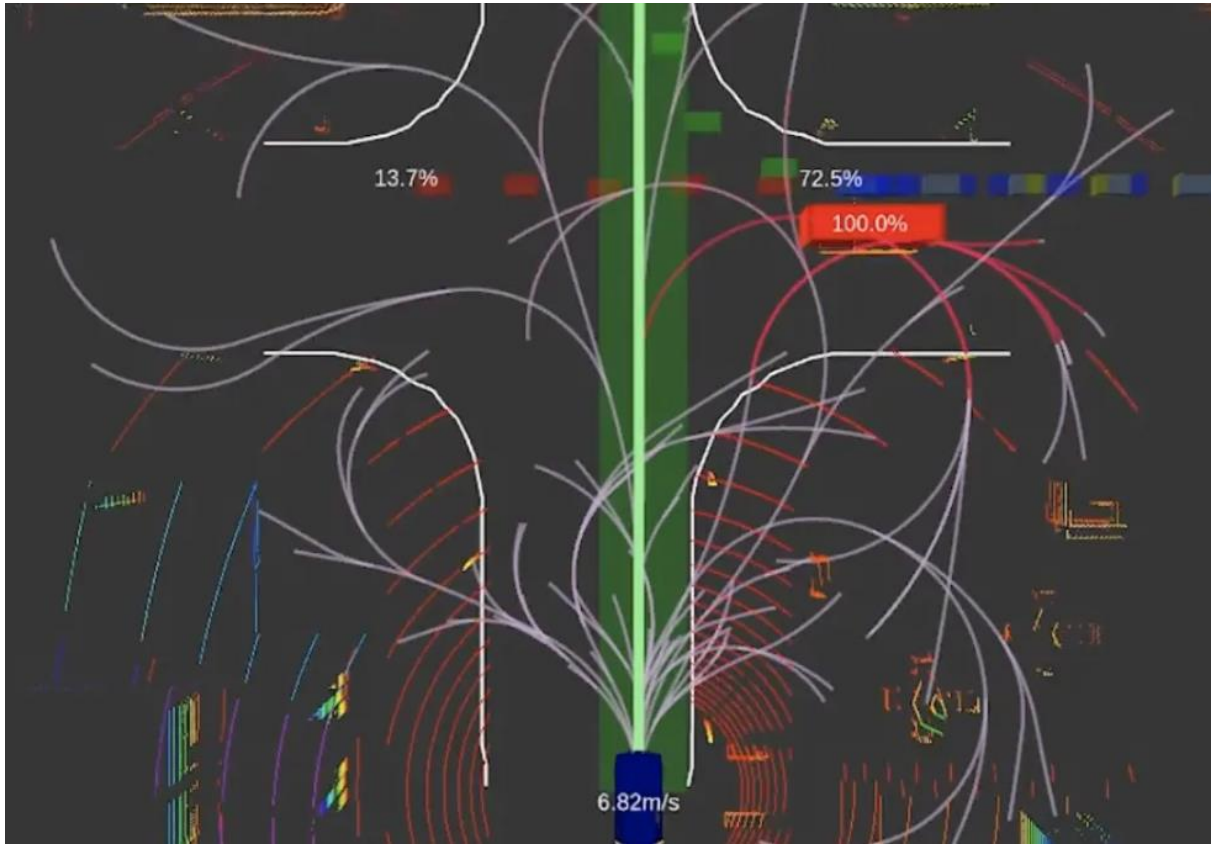
- Pseudo-code

Algorithm 1 A*(G, s, t)

```
open ← MinHeap()
closed ← Set()
predecessors ← Dict()
costs ← Dict()
open.push(s, 0)
costs[s] ← 0
while !open.isEmpty() do
    u ← open.pop()
    uCost ← costs[u]
    if isGoal(u) then
        return extractPath(u, predecessors)
    for all v ∈ u.successors do
        if v ∈ closed then
            continue
        uvCost ← edgeCost(G, u, v)
        if v ∈ open then
            if uCost + uvCost + h(v) < open[v] then
                open[v] ← uCost + uvCost + h(v)
                costs[v] ← uCost + uvCost
                predecessors[v] ← u
        else
            open.push(v, uCost + uvCost + h(v))
            costs[v] ← uCost + uvCost
            predecessors[v] ← u
    closed.add(u)
```

Motion Planning

- Find feasible, comfortable, safe and fast **vehicle path/trajectory**



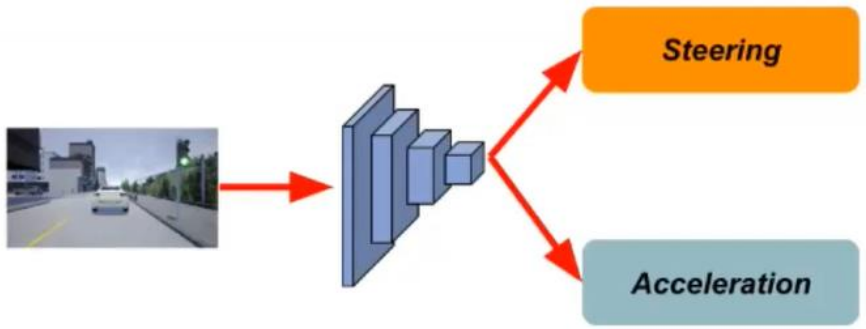
Outputs

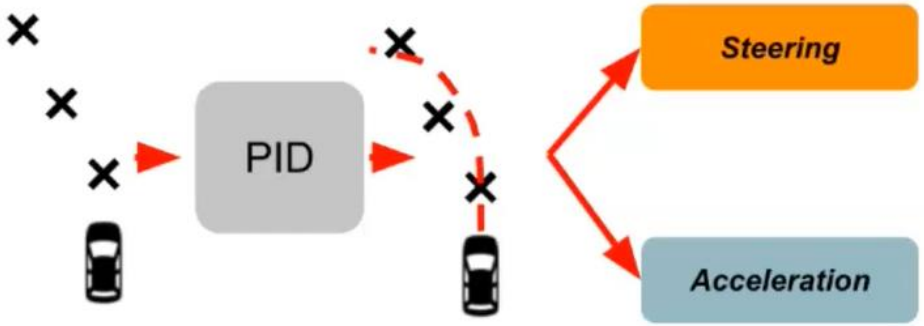
Actuation

- Directly predict steering/acceleration
- Or trajectory of way points

- Strengths:
  - Simple, well-studied

- Weaknesses:
  - Hard to incorporate prior knowledge
  - Lacks interpretability





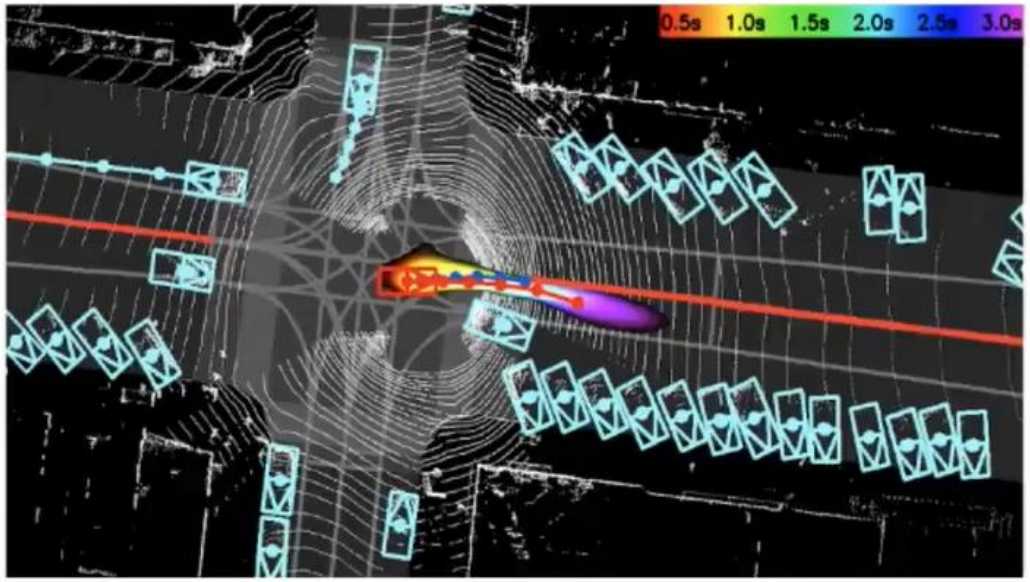
Outputs

Costs

- Predict the cost of a trajectory and then plan a trajectory with the minimal cost
  - E.g., predict cost of SDV being at a particular space-time location

- Strengths:
  - Easy to encode prior knowledge

- Weaknesses:
  - No direct supervision



Outputs

Affordance

- actionable, interpretable, physical quantities that describe “what the environment offers the car.”

- Distance to nearest centerline
- Distance to nearest agent
- Angle relative to lane
- Whether there’s a crosswalk
- Whether you can turn left safely
- Time to collision
- … …

- Advantages:
  - Fully supervised learning
  - Interpretable
  - Modular & Safe

- Limitations
  - Needs handcrafted design of affordances

    If you miss an important one → planning fails

Outputs

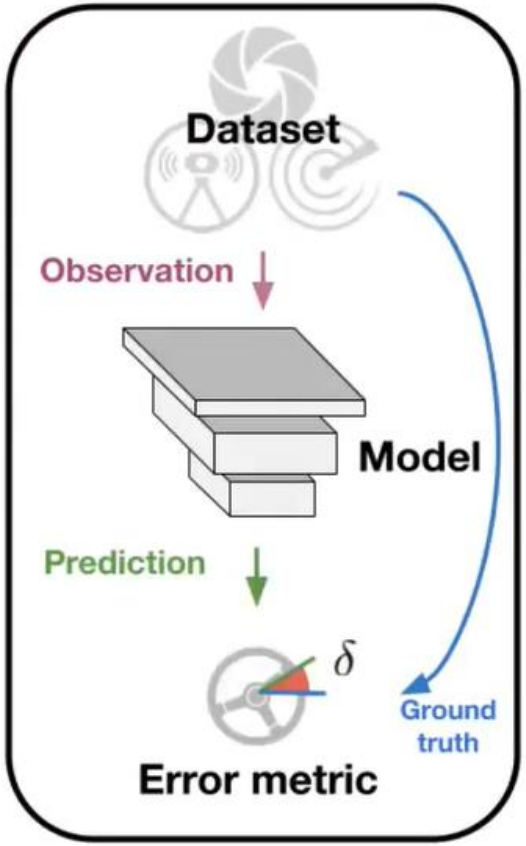
Auxiliary objectives

- Auxiliary objectives to improve planning
  - Object detection
  - Segmentation
  - Depth completion
  - Optical flow
- Train the model with **auxiliary objectives as supervised signals**
  - alongside the main planning task
- Forces the model to learn low-level visual structure (like lanes, objects, depth)
- Once trained, discard the auxiliary outputs — only the main planning output (affordances/cost/trajectory) is used for real-world motion planning.”

Behavior Cloning

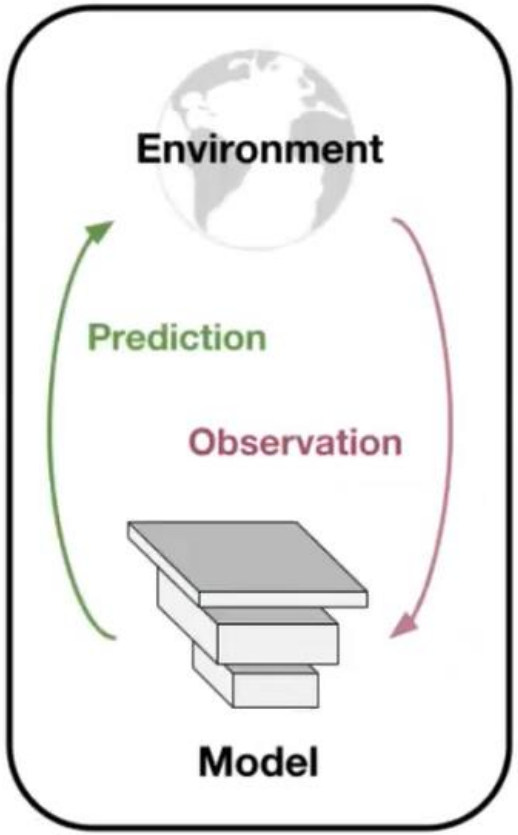
- Open-loop learning: Policy's actions never executed
- Behavior cloning:
  - A type of open-loop learning
  - Policy observes expert states
  - Supervised to imitate expert actions:

    L2 loss between policy and expert actions
- Strengths:
  - Simple to implement
  - High learning efficiency
- Weakness:
  - Distribution shift (training vs inference)



Close-loop Learning

- Closed-loop learning:
  - Execute policy's actions and see new observations
  - Requires learning in the real world or realistic simulation
- Advantage:
  - Less distribution shift between training and inference
- Challenge: What is the supervision for new states visited by the model?

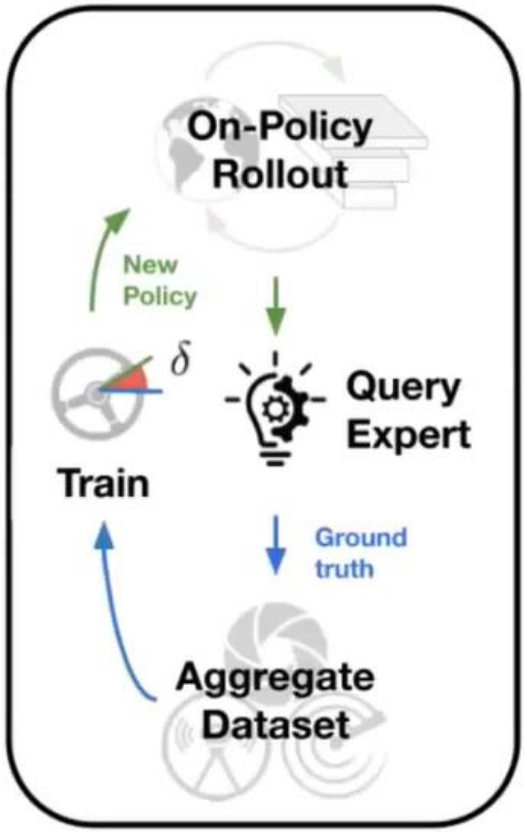


Interactive Expert

- Use an interactive expert to label new states the policy visits
- Retrain policy and new examples and iterate

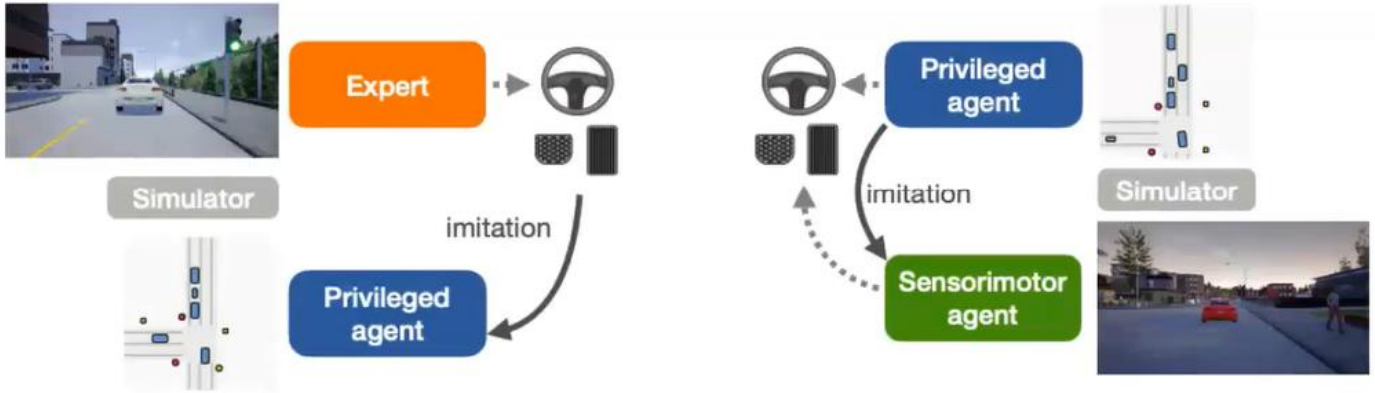
- Strengths:
  - Trains policy to recover from mistake with expert supervision

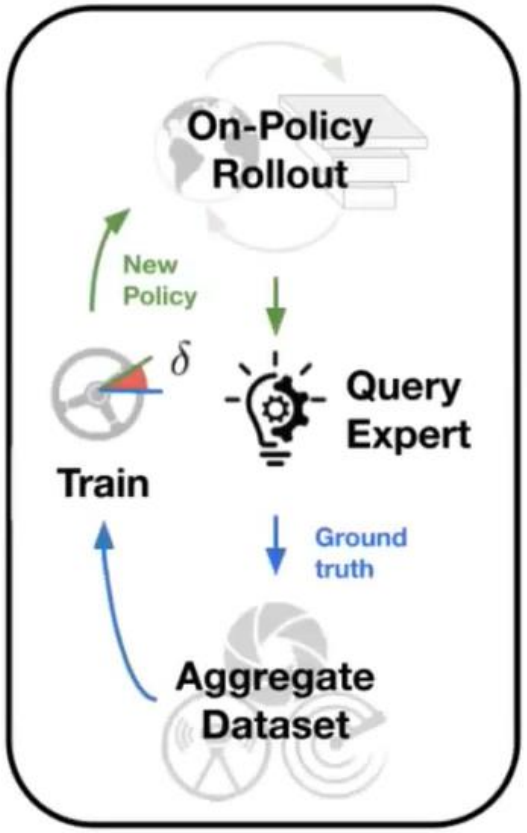
- Weaknesses:
  - Interactive expert is rarely available



Interactive Expert

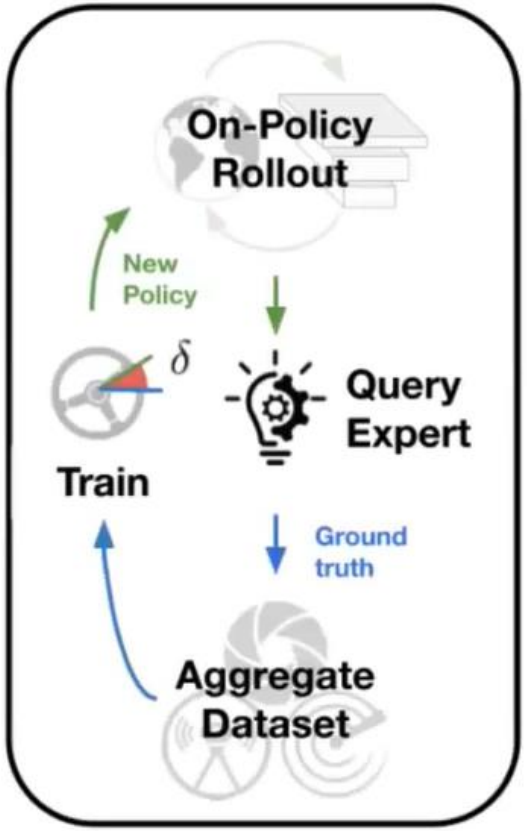
- Policy distillation
  - Train/build a privileged policy
  - Distill privileged policy into learner policy





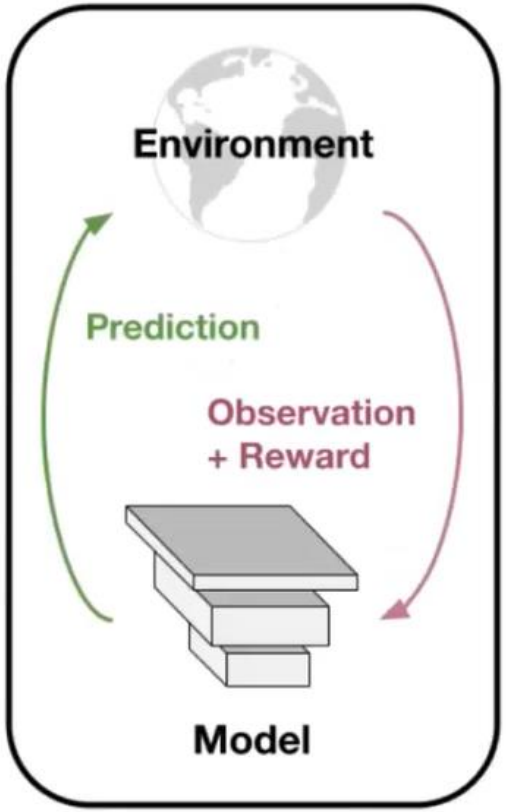
Interactive Expert

- Policy distillation
  - Train/build a privileged policy
  - Distill privileged policy into learner policy
- Strengths:
  - Privileged policy can act as an expert to supervise the learner
- Weakness
  - Learner is upper-bounded by the potentially suboptimal expert



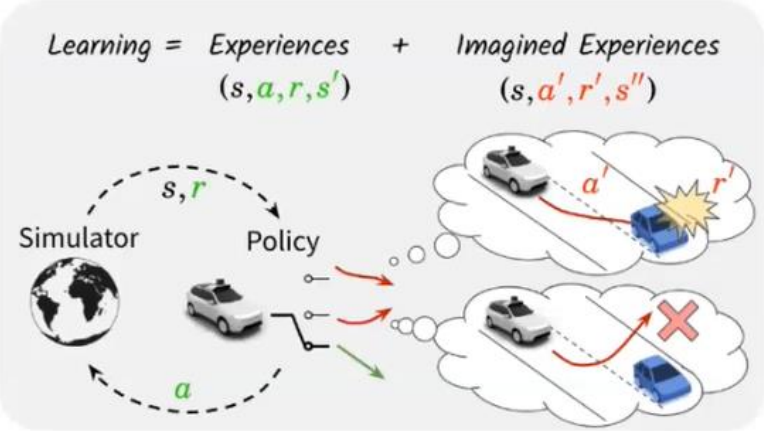
Reinforcement Learning

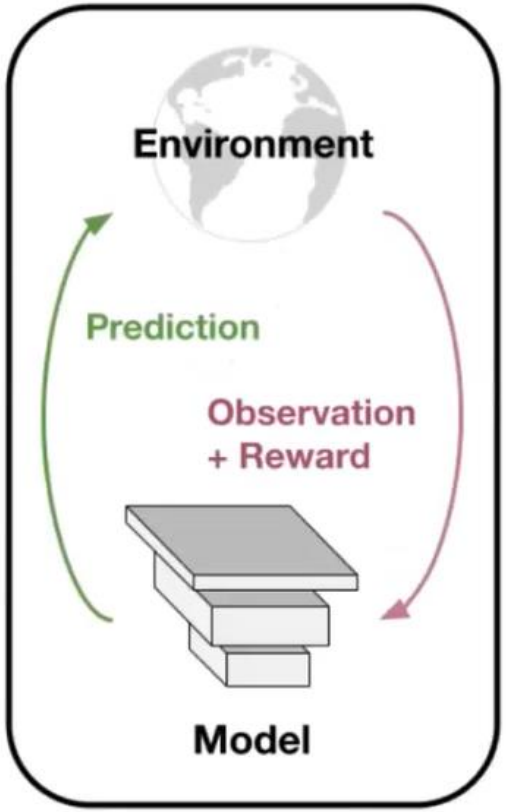
- Encode explicit learning signal through reward design
- RL in the real world
  - Eliminates domain gap, but difficult to scale safely
- RL in simulation
  - Realistic simulation is important to minimize domain gap
  - Sample efficiency can be prohibitively low even in simulation



Reinforcement Learning

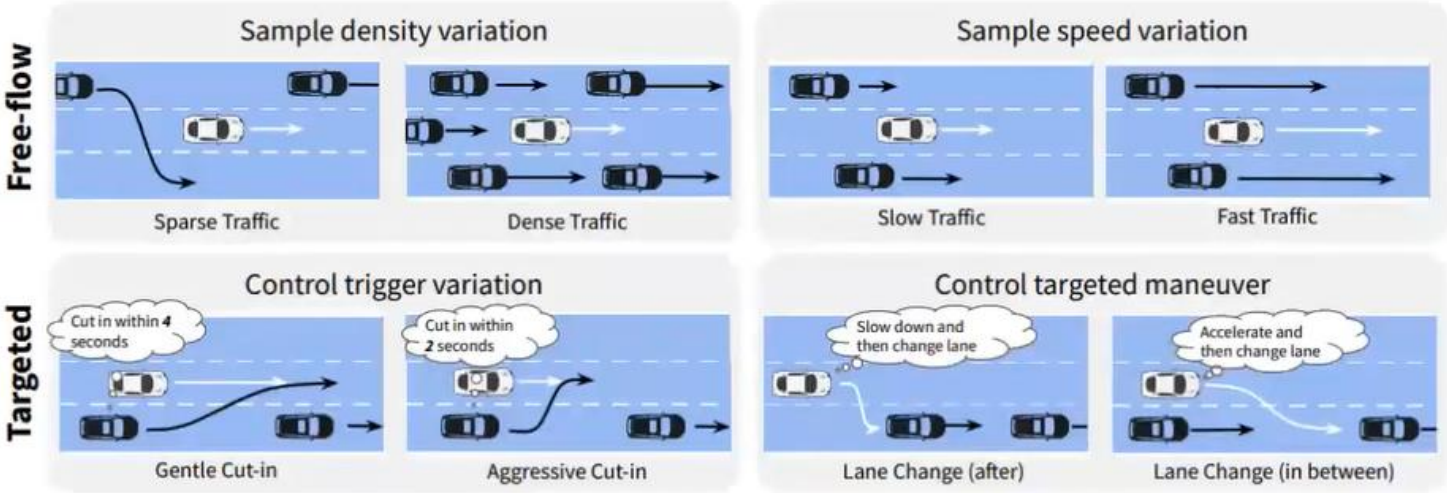
- Improving efficiency
  - Using approximate world model for more efficient simulations
  - Learning policies which plan in trajectory space for easier long-term reasoning

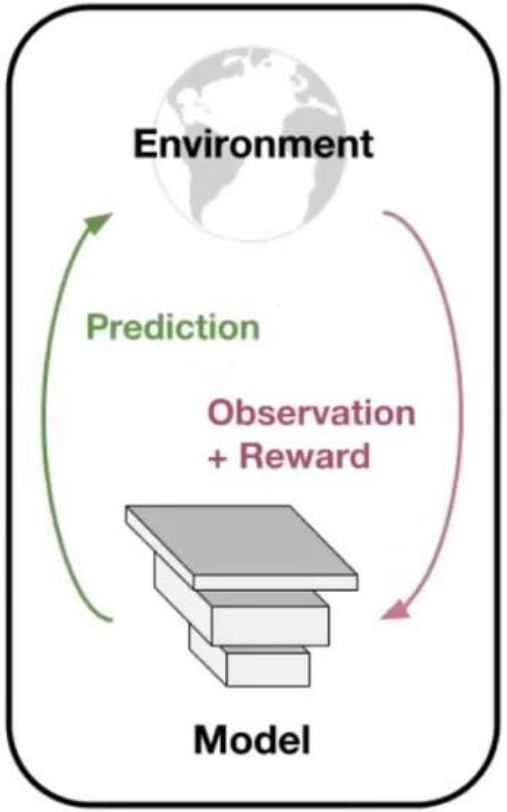




Reinforcement Learning

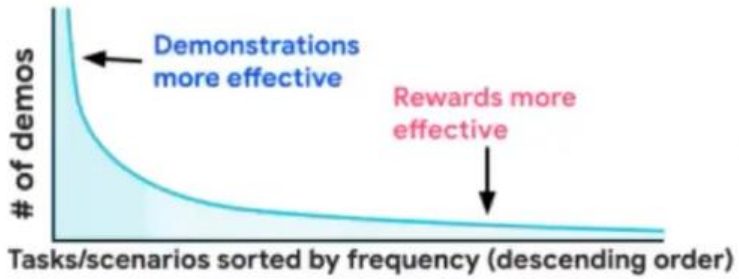
- Designing the closed-loop benchmark
- Incorporating targeted safety-critical scenarios, scaling up behavioral diversity

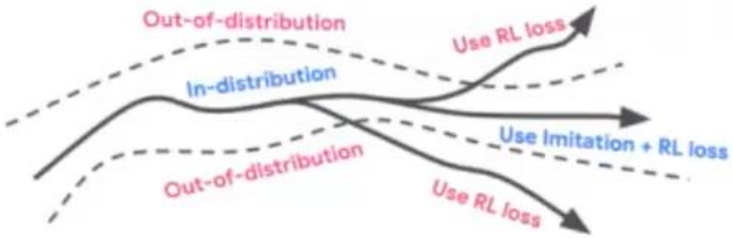


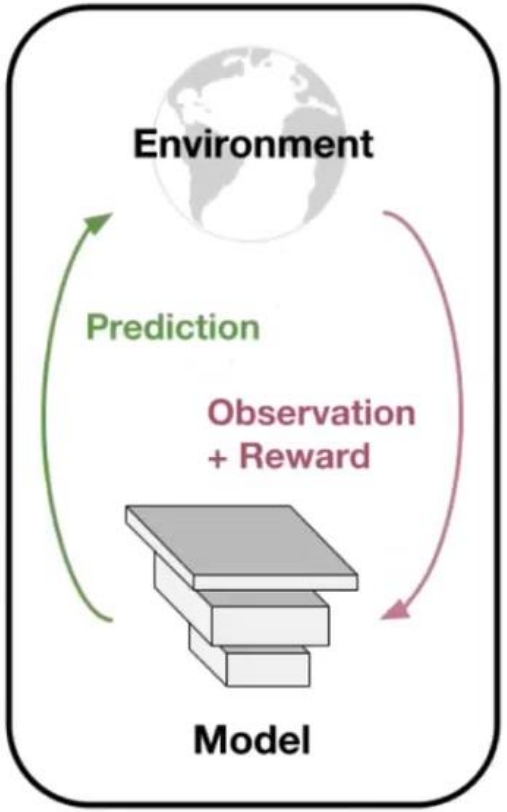


Reinforcement Learning

- Incorporating real data
  - Using imitate learning pretrained model for initialization to guide exploration
  - Using imitate learning loss plus a sparse reward to learn realistic behaviors without manual reward shaping



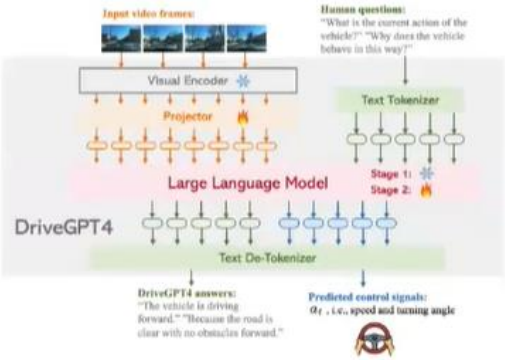




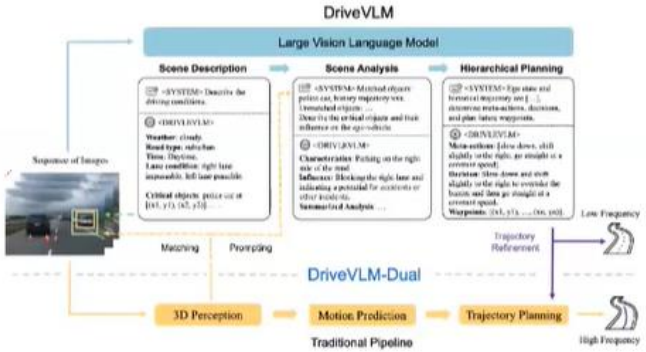
Foundation Models for Motion Planning

LLM + Tool Use[1]

LLM + Adapters[2]



VLM[3]



[1] A Language Agent for Autonomous Driving [Mao et. al. 2023]
[2] DriveGPT4: Interpretable End-to-end Autonomous Driving via Large Language Model [Xu et. al. 2024]
[3] DriveVLM: The Convergence of Autonomous Driving and Large Vision-Language Models [Tian et. al. 2024]